

My Project

Generated by Doxygen 1.13.2

1 Hierarchical Index	1
1.1 Class Hierarchy	1
2 Class Index	3
2.1 Class List	3
3 File Index	5
3.1 File List	5
4 Class Documentation	7
4.1 duom Class Reference	7
4.1.1 Detailed Description	8
4.1.2 Constructor & Destructor Documentation	8
4.1.2.1 duom() [1/6]	8
4.1.2.2 duom() [2/6]	8
4.1.2.3 duom() [3/6]	8
4.1.2.4 duom() [4/6]	8
4.1.2.5 ~duom()	9
4.1.2.6 duom() [5/6]	9
4.1.2.7 duom() [6/6]	9
4.1.3 Member Function Documentation	9
4.1.3.1 abstractClassFunction()	9
4.1.3.2 addPazymiai()	9
4.1.3.3 getExam()	9
4.1.3.4 getMark()	10
4.1.3.5 getPav()	10
4.1.3.6 getPazymiai()	10
4.1.3.7 getPazymiai_at()	10
4.1.3.8 getVar()	10
4.1.3.9 getVid_med()	10
4.1.3.10 isClear()	10
4.1.3.11 operator=() [1/2]	11
4.1.3.12 operator=() [2/2]	11
4.1.3.13 operator==()	11
4.1.3.14 setExam()	11
4.1.3.15 setMark()	11
4.1.3.16 setPazymiai()	11
4.1.3.17 setVid_med()	12
4.1.4 Friends And Related Symbol Documentation	12
4.1.4.1 operator<<	12
4.1.4.2 operator==	12
4.1.4.3 operator>>	12
4.2 human Class Reference	12

4.2.1 Detailed Description	13
4.2.2 Member Function Documentation	13
4.2.2.1 abstractClassFunction()	13
4.2.3 Member Data Documentation	13
4.2.3.1 pav	13
4.2.3.2 var	13
4.3 StudentaiTest Class Reference	13
4.3.1 Detailed Description	14
4.3.2 Member Function Documentation	14
4.3.2.1 createSampleStudent()	14
4.3.2.2 SetUp()	14
4.3.3 Member Data Documentation	14
4.3.3.1 testNd	14
4.4 temp Struct Reference	14
4.4.1 Detailed Description	15
4.4.2 Member Data Documentation	15
4.4.2.1 exam	15
4.4.2.2 mark	15
4.4.2.3 pav	15
4.4.2.4 pazymiai	15
4.4.2.5 var	15
4.4.2.6 vid_med	15
4.5 Vector< T > Class Template Reference	16
4.5.1 Detailed Description	17
4.5.2 Member Typedef Documentation	17
4.5.2.1 const_iterator	17
4.5.2.2 const_pointer	18
4.5.2.3 const_reference	18
4.5.2.4 const_reverse_iterator	18
4.5.2.5 difference_type	18
4.5.2.6 iterator	18
4.5.2.7 pointer	18
4.5.2.8 reference	18
4.5.2.9 reverse_iterator	19
4.5.2.10 size_type	19
4.5.2.11 value_type	19
4.5.3 Constructor & Destructor Documentation	19
4.5.3.1 Vector() [1/7]	19
4.5.3.2 Vector() [2/7]	19
4.5.3.3 Vector() [3/7]	19
4.5.3.4 Vector() [4/7]	20
4.5.3.5 Vector() [5/7]	20

4.5.3.6 Vector() [6/7]	20
4.5.3.7 Vector() [7/7]	20
4.5.3.8 ~Vector()	20
4.5.4 Member Function Documentation	21
4.5.4.1 assign() [1/3]	21
4.5.4.2 assign() [2/3]	21
4.5.4.3 assign() [3/3]	21
4.5.4.4 at() [1/2]	21
4.5.4.5 at() [2/2]	21
4.5.4.6 back() [1/2]	22
4.5.4.7 back() [2/2]	22
4.5.4.8 begin() [1/2]	22
4.5.4.9 begin() [2/2]	22
4.5.4.10 capacity()	22
4.5.4.11 cbegin()	22
4.5.4.12 cend()	23
4.5.4.13 clear()	23
4.5.4.14 crbegin()	23
4.5.4.15 crend()	23
4.5.4.16 data() [1/2]	23
4.5.4.17 data() [2/2]	23
4.5.4.18 emplace()	24
4.5.4.19 emplace_back()	24
4.5.4.20 empty()	24
4.5.4.21 end() [1/2]	24
4.5.4.22 end() [2/2]	24
4.5.4.23 erase() [1/2]	25
4.5.4.24 erase() [2/2]	25
4.5.4.25 front() [1/2]	25
4.5.4.26 front() [2/2]	25
4.5.4.27 getMemory_reallocated()	25
4.5.4.28 insert() [1/3]	25
4.5.4.29 insert() [2/3]	26
4.5.4.30 insert() [3/3]	26
4.5.4.31 max_size()	26
4.5.4.32 operator=() [1/3]	26
4.5.4.33 operator=() [2/3]	26
4.5.4.34 operator=() [3/3]	27
4.5.4.35 operator[]() [1/2]	27
4.5.4.36 operator[]() [2/2]	27
4.5.4.37 pop_back()	27
4.5.4.38 push_back() [1/2]	27

4.5.4.39 push_back() [2/2]	28
4.5.4.40 rbegin() [1/2]	28
4.5.4.41 rbegin() [2/2]	28
4.5.4.42 rend() [1/2]	28
4.5.4.43 rend() [2/2]	28
4.5.4.44 reserve()	29
4.5.4.45 resize() [1/2]	29
4.5.4.46 resize() [2/2]	29
4.5.4.47 shrink_to_fit()	29
4.5.4.48 size()	29
4.5.4.49 swap()	29
5 File Documentation	31
5.1 main.cpp File Reference	31
5.1.1 Function Documentation	31
5.1.1.1 main()	31
5.2 main.cpp	31
5.3 program.cpp File Reference	32
5.3.1 Function Documentation	32
5.3.1.1 average()	32
5.3.1.2 check_menu()	32
5.3.1.3 class_write_test()	33
5.3.1.4 compare()	33
5.3.1.5 median()	33
5.3.1.6 menu()	33
5.3.1.7 method_test()	33
5.3.1.8 print_answers_console()	33
5.3.1.9 print_answers_to_file()	34
5.3.1.10 print_data_to_file()	34
5.3.1.11 random_grades()	34
5.3.1.12 random_names_grades()	34
5.3.1.13 read_file()	34
5.3.1.14 read_from_console()	35
5.3.1.15 read_names_from_console()	35
5.3.1.16 sorting()	35
5.3.1.17 vid_med_calc()	35
5.3.2 Variable Documentation	35
5.3.2.1 vardai	35
5.4 program.cpp	36
5.5 std.h File Reference	43
5.6 std.h	43
5.7 Studentai_tests.cpp File Reference	43

5.8 Studentai_tests.cpp	43
5.9 template.h File Reference	45
5.9.1 Function Documentation	46
5.9.1.1 average()	46
5.9.1.2 check_menu()	46
5.9.1.3 class_write_test()	46
5.9.1.4 compare()	46
5.9.1.5 median()	46
5.9.1.6 menu()	47
5.9.1.7 method_test()	47
5.9.1.8 print_answers_console()	47
5.9.1.9 print_answers_to_file()	47
5.9.1.10 print_data_to_file()	47
5.9.1.11 random_grades()	48
5.9.1.12 random_names_grades()	48
5.9.1.13 read_file()	48
5.9.1.14 read_from_console()	48
5.9.1.15 read_names_from_console()	48
5.9.1.16 sorting()	49
5.9.1.17 vid_med_calc()	49
5.9.2 Variable Documentation	49
5.9.2.1 test_file_location	49
5.10 template.h	49
5.11 Vector.h File Reference	52
5.11.1 Function Documentation	53
5.11.1.1 destroy_range()	53
5.11.1.2 operator!=(())	53
5.11.1.3 operator<()	54
5.11.1.4 operator<=()	54
5.11.1.5 operator==()	54
5.11.1.6 operator>()	54
5.11.1.7 operator>=()	54
5.11.1.8 swap()	55
5.12 Vector.h	55

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

human	12
duom	7
temp	14
testing::Test	
StudentaiTest	13
Vector< T >	16

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

duom	7
human	12
StudentaiTest	13
temp	14
Vector< T >	16

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

main.cpp	31
program.cpp	32
std.h	43
Studentai_tests.cpp	43
template.h	45
Vector.h	52

Chapter 4

Class Documentation

4.1 duom Class Reference

```
#include <template.h>
```

Inheritance diagram for duom:

Collaboration diagram for duom:

Public Member Functions

- void [abstractClassFunction](#) () override
- [duom](#) (temp &a)
- [duom](#) (string var, string pav)
- [duom](#) (string var, string pav, [Vector](#)< int > &pazymiai, int exam)
- [duom](#) ()
- [~duom](#) ()
- [duom](#) (const [duom](#) &to_copy)
- [duom](#) ([duom](#) &&to_move) noexcept
- [duom](#) & [operator=](#) (const [duom](#) &to_copy)
- [duom](#) & [operator=](#) ([duom](#) &&to_move) noexcept
- bool [isClear](#) () noexcept
- bool [operator==](#) ([duom](#) &&to_compare) noexcept
- string [getVar](#) () const
- string [getPav](#) () const
- int [getPazymiai_at](#) (int i) const
- [Vector](#)< int > [getPazymiai](#) () const
- int [getExam](#) () const
- double [getVid_med](#) () const
- double [getMark](#) () const
- void [setPazymiai](#) ([Vector](#)< int > &pazymiai)
- void [addPazymiai](#) (int grade)
- void [setExam](#) (int exam)
- void [setVid_med](#) (double vid_med)
- void [setMark](#) (double mark)

Friends

- bool `operator==` (const `duom` &a, const `duom` &b) noexcept
- std::ifstream & `operator>>` (std::ifstream &in, `duom` &student)
- std::ostream & `operator<<` (std::ostream &out, `duom` &student)

4.1.1 Detailed Description

Definition at line 41 of file [template.h](#).

4.1.2 Constructor & Destructor Documentation

4.1.2.1 `duom()` [1/6]

```
duom::duom (  
    temp & a) [inline]
```

Definition at line 52 of file [template.h](#).

Here is the caller graph for this function:

4.1.2.2 `duom()` [2/6]

```
duom::duom (  
    string var,  
    string pav) [inline]
```

Definition at line 61 of file [template.h](#).

4.1.2.3 `duom()` [3/6]

```
duom::duom (  
    string var,  
    string pav,  
    Vector< int > & pazymiai,  
    int exam) [inline]
```

Definition at line 66 of file [template.h](#).

4.1.2.4 `duom()` [4/6]

```
duom::duom () [inline]
```

Definition at line 73 of file [template.h](#).

4.1.2.5 ~duom()

```
duom::~~duom () [inline]
```

Definition at line 75 of file [template.h](#).

4.1.2.6 duom() [5/6]

```
duom::duom (
    const duom & to_copy) [inline]
```

Definition at line 85 of file [template.h](#).

Here is the call graph for this function:

4.1.2.7 duom() [6/6]

```
duom::duom (
    duom && to_move) [inline], [noexcept]
```

move naudojam tik su elementais saugomais heap'e

Definition at line 95 of file [template.h](#).

Here is the call graph for this function:

4.1.3 Member Function Documentation

4.1.3.1 abstractClassFunction()

```
void duom::abstractClassFunction () [inline], [override], [virtual]
```

Implements [human](#).

Definition at line 44 of file [template.h](#).

4.1.3.2 addPazymiai()

```
void duom::addPazymiai (
    int grade) [inline]
```

Definition at line 225 of file [template.h](#).

4.1.3.3 getExam()

```
int duom::getExam () const [inline]
```

Definition at line 208 of file [template.h](#).

4.1.3.4 getMark()

```
double duom::getMark () const [inline]
```

Definition at line 216 of file [template.h](#).

4.1.3.5 getPav()

```
string duom::getPav () const [inline]
```

Definition at line 196 of file [template.h](#).

4.1.3.6 getPazymiai()

```
Vector< int > duom::getPazymiai () const [inline]
```

Definition at line 204 of file [template.h](#).

Here is the caller graph for this function:

4.1.3.7 getPazymiai_at()

```
int duom::getPazymiai_at (  
    int i) const [inline]
```

Definition at line 200 of file [template.h](#).

4.1.3.8 getVar()

```
string duom::getVar () const [inline]
```

Definition at line 192 of file [template.h](#).

4.1.3.9 getVid_med()

```
double duom::getVid_med () const [inline]
```

Definition at line 212 of file [template.h](#).

4.1.3.10 isClear()

```
bool duom::isClear () [inline], [noexcept]
```

Definition at line 137 of file [template.h](#).

Here is the call graph for this function: Here is the caller graph for this function:

4.1.3.11 operator=() [1/2]

```
duom & duom::operator= (
    const duom & to_copy) [inline]
```

Definition at line 109 of file [template.h](#).

Here is the call graph for this function:

4.1.3.12 operator=() [2/2]

```
duom & duom::operator= (
    duom && to_move) [inline], [noexcept]
```

move naudojam tik su elementais saugomais heap'e

Definition at line 121 of file [template.h](#).

Here is the call graph for this function:

4.1.3.13 operator==()

```
bool duom::operator== (
    duom && to_compare) [inline], [noexcept]
```

Definition at line 143 of file [template.h](#).

Here is the call graph for this function:

4.1.3.14 setExam()

```
void duom::setExam (
    int exam) [inline]
```

Definition at line 232 of file [template.h](#).

4.1.3.15 setMark()

```
void duom::setMark (
    double mark) [inline]
```

Definition at line 246 of file [template.h](#).

4.1.3.16 setPazymiai()

```
void duom::setPazymiai (
    Vector< int > & pazymiai) [inline]
```

Definition at line 221 of file [template.h](#).

4.1.3.17 setVid_med()

```
void duom::setVid_med (
    double vid_med) [inline]
```

Definition at line 239 of file [template.h](#).

4.1.4 Friends And Related Symbol Documentation

4.1.4.1 operator<<

```
std::ostream & operator<< (
    std::ostream & out,
    duom & student) [friend]
```

Definition at line 187 of file [template.h](#).

4.1.4.2 operator==

```
bool operator== (
    const duom & a,
    const duom & b) [friend]
```

Definition at line 151 of file [template.h](#).

4.1.4.3 operator>>

```
std::ifstream & operator>> (
    std::ifstream & in,
    duom & student) [friend]
```

Definition at line 159 of file [template.h](#).

The documentation for this class was generated from the following file:

- [template.h](#)

4.2 human Class Reference

```
#include <template.h>
```

Inheritance diagram for human:

Public Member Functions

- virtual void [abstractClassFunction](#) ()=0

Protected Attributes

- string `var` = ""
- string `pav` = ""

4.2.1 Detailed Description

Definition at line 30 of file [template.h](#).

4.2.2 Member Function Documentation

4.2.2.1 abstractClassFunction()

```
virtual void human::abstractClassFunction () [pure virtual]
```

Implemented in [duom](#).

4.2.3 Member Data Documentation

4.2.3.1 pav

```
string human::pav = "" [protected]
```

Definition at line 37 of file [template.h](#).

4.2.3.2 var

```
string human::var = "" [protected]
```

Definition at line 36 of file [template.h](#).

The documentation for this class was generated from the following file:

- [template.h](#)

4.3 StudentaiTest Class Reference

Inheritance diagram for StudentaiTest:

Collaboration diagram for StudentaiTest:

Protected Member Functions

- void [SetUp](#) () override
- [duom createSampleStudent](#) ()

Protected Attributes

- [Vector](#)< int > [testNd](#)

4.3.1 Detailed Description

Definition at line 4 of file [Studentai_tests.cpp](#).

4.3.2 Member Function Documentation

4.3.2.1 createSampleStudent()

```
duom StudentaiTest::createSampleStudent () [inline], [protected]
```

Definition at line 11 of file [Studentai_tests.cpp](#).

4.3.2.2 SetUp()

```
void StudentaiTest::SetUp () [inline], [override], [protected]
```

Definition at line 6 of file [Studentai_tests.cpp](#).

4.3.3 Member Data Documentation

4.3.3.1 testNd

```
Vector<int> StudentaiTest::testNd [protected]
```

Definition at line 9 of file [Studentai_tests.cpp](#).

The documentation for this class was generated from the following file:

- [Studentai_tests.cpp](#)

4.4 temp Struct Reference

```
#include <template.h>
```

Collaboration diagram for temp:

Public Attributes

- string [var](#) ="test"
- string [pav](#) ="test"
- [Vector](#)< int > [pazymiai](#)
- int [exam](#) =0
- double [vid_med](#) =0
- double [mark](#)

4.4.1 Detailed Description

Definition at line 20 of file [template.h](#).

4.4.2 Member Data Documentation

4.4.2.1 exam

```
int temp::exam =0
```

Definition at line 25 of file [template.h](#).

4.4.2.2 mark

```
double temp::mark
```

Definition at line 27 of file [template.h](#).

4.4.2.3 pav

```
string temp::pav ="test"
```

Definition at line 23 of file [template.h](#).

4.4.2.4 pazymiai

```
Vector<int> temp::pazymiai
```

Definition at line 24 of file [template.h](#).

4.4.2.5 var

```
string temp::var ="test"
```

Definition at line 22 of file [template.h](#).

4.4.2.6 vid_med

```
double temp::vid_med =0
```

Definition at line 26 of file [template.h](#).

The documentation for this struct was generated from the following file:

- [template.h](#)

4.5 Vector< T > Class Template Reference

```
#include <Vector.h>
```

Public Types

- using [value_type](#) = T
- using [pointer](#) = T*
- using [const_pointer](#) = const T*
- using [reference](#) = T&
- using [const_reference](#) = const T&
- using [size_type](#) = std::size_t
- using [difference_type](#) = std::ptrdiff_t
- using [iterator](#) = T*
- using [const_iterator](#) = const T*
- using [reverse_iterator](#) = std::reverse_iterator<[iterator](#)>
- using [const_reverse_iterator](#) = std::reverse_iterator<[const_iterator](#)>

Public Member Functions

- [Vector](#) () noexcept=default
- [Vector](#) ([size_type](#) count)
- [Vector](#) ([size_type](#) count, [const_reference](#) value)
- template<typename InputIt, typename = std::enable_if_t< std::is_base_of_v<std::input_iterator_tag, typename std::iterator_traits<↔
InputIt>::iterator_category> >>>
 [Vector](#) (InputIt first, InputIt last)
- [Vector](#) (const [Vector](#) &other)
- [Vector](#) ([Vector](#) &&other) noexcept
- [Vector](#) (std::initializer_list< T > init)
- [~Vector](#) ()
- [Vector](#) & [operator=](#) (const [Vector](#) &other)
- [Vector](#) & [operator=](#) ([Vector](#) &&other) noexcept
- [Vector](#) & [operator=](#) (std::initializer_list< T > ilist)
- void [assign](#) ([size_type](#) count, const T &value)
- template<typename InputIt, typename = std::enable_if_t< std::is_base_of_v<std::input_iterator_tag, typename std::iterator_traits<↔
InputIt>::iterator_category> >>>
 void [assign](#) (InputIt first, InputIt last)
- void [assign](#) (std::initializer_list< T > ilist)
- int [getMemory_reallocated](#) ()
- [reference](#) at ([size_type](#) pos)
- [const_reference](#) at ([size_type](#) pos) const
- [reference](#) operator[] ([size_type](#) pos)
- [const_reference](#) operator[] ([size_type](#) pos) const
- [reference](#) front ()
- [const_reference](#) front () const
- [reference](#) back ()
- [const_reference](#) back () const
- T * [data](#) () noexcept
- const T * [data](#) () const noexcept
- [iterator](#) begin () noexcept
- [const_iterator](#) begin () const noexcept
- [const_iterator](#) cbegin () const noexcept

- [iterator end](#) () noexcept
- [const_iterator end](#) () const noexcept
- [const_iterator cend](#) () const noexcept
- [reverse_iterator rbegin](#) () noexcept
- [const_reverse_iterator rbegin](#) () const noexcept
- [const_reverse_iterator crbegin](#) () const noexcept
- [reverse_iterator rend](#) () noexcept
- [const_reverse_iterator rend](#) () const noexcept
- [const_reverse_iterator crend](#) () const noexcept
- [bool empty](#) () const noexcept
- [size_type size](#) () const noexcept
- [size_type max_size](#) () const noexcept
- [void reserve](#) ([size_type](#) new_cap)
- [size_type capacity](#) () const noexcept
- [void shrink_to_fit](#) ()
- [void clear](#) () noexcept
- [iterator insert](#) ([const_iterator](#) pos, const T &value)
- [iterator insert](#) ([const_iterator](#) pos, T &&value)
- [iterator insert](#) ([const_iterator](#) pos, [size_type](#) count, const T &value)
- [template<typename... Args>](#)
[iterator emplace](#) ([const_iterator](#) pos, Args &&... args)
- [iterator erase](#) ([const_iterator](#) pos)
- [iterator erase](#) ([const_iterator](#) first, [const_iterator](#) last)
- [void push_back](#) (const T &value)
- [void push_back](#) (T &&value)
- [template<typename... Args>](#)
[reference emplace_back](#) (Args &&... args)
- [void pop_back](#) ()
- [void resize](#) ([size_type](#) count)
- [void resize](#) ([size_type](#) count, const [value_type](#) &value)
- [void swap](#) ([Vector](#) &other) noexcept

4.5.1 Detailed Description

```
template<typename T>
class Vector< T >
```

Definition at line 24 of file [Vector.h](#).

4.5.2 Member Typedef Documentation

4.5.2.1 const_iterator

```
template<typename T>
using Vector< T >::const_iterator = const T*
```

Definition at line 38 of file [Vector.h](#).

4.5.2.2 const_pointer

```
template<typename T>
using Vector< T >::const_pointer = const T*
```

Definition at line 29 of file [Vector.h](#).

4.5.2.3 const_reference

```
template<typename T>
using Vector< T >::const_reference = const T&
```

Definition at line 31 of file [Vector.h](#).

4.5.2.4 const_reverse_iterator

```
template<typename T>
using Vector< T >::const_reverse_iterator = std::reverse_iterator<const_iterator>
```

Definition at line 40 of file [Vector.h](#).

4.5.2.5 difference_type

```
template<typename T>
using Vector< T >::difference_type = std::ptrdiff_t
```

Definition at line 33 of file [Vector.h](#).

4.5.2.6 iterator

```
template<typename T>
using Vector< T >::iterator = T*
```

Definition at line 37 of file [Vector.h](#).

4.5.2.7 pointer

```
template<typename T>
using Vector< T >::pointer = T*
```

Definition at line 28 of file [Vector.h](#).

4.5.2.8 reference

```
template<typename T>
using Vector< T >::reference = T&
```

Definition at line 30 of file [Vector.h](#).

4.5.2.9 reverse_iterator

```
template<typename T>
using Vector< T >::reverse_iterator = std::reverse_iterator<iterator>
```

Definition at line 39 of file [Vector.h](#).

4.5.2.10 size_type

```
template<typename T>
using Vector< T >::size_type = std::size_t
```

Definition at line 32 of file [Vector.h](#).

4.5.2.11 value_type

```
template<typename T>
using Vector< T >::value_type = T
```

Definition at line 27 of file [Vector.h](#).

4.5.3 Constructor & Destructor Documentation

4.5.3.1 Vector() [1/7]

```
template<typename T>
Vector< T >::Vector () [default], [noexcept]
```

Here is the caller graph for this function:

4.5.3.2 Vector() [2/7]

```
template<typename T>
Vector< T >::Vector (
    size_type count) [inline], [explicit]
```

Definition at line 118 of file [Vector.h](#).

Here is the call graph for this function:

4.5.3.3 Vector() [3/7]

```
template<typename T>
Vector< T >::Vector (
    size_type count,
    const_reference value) [inline]
```

Definition at line 137 of file [Vector.h](#).

Here is the call graph for this function:

4.5.3.4 Vector() [4/7]

```
template<typename T>
template<typename InputIt, typename = std::enable_if_t< std::is_base_of_v<std::input_iterator,
_tag, typename std::iterator_traits<InputIt>::iterator_category> >>
Vector< T >::Vector (
    InputIt first,
    InputIt last) [inline]
```

Definition at line 156 of file [Vector.h](#).

Here is the call graph for this function:

4.5.3.5 Vector() [5/7]

```
template<typename T>
Vector< T >::Vector (
    const Vector< T > & other) [inline]
```

Definition at line 188 of file [Vector.h](#).

Here is the call graph for this function:

4.5.3.6 Vector() [6/7]

```
template<typename T>
Vector< T >::Vector (
    Vector< T > && other) [inline], [noexcept]
```

Definition at line 204 of file [Vector.h](#).

Here is the call graph for this function:

4.5.3.7 Vector() [7/7]

```
template<typename T>
Vector< T >::Vector (
    std::initializer_list< T > init) [inline]
```

Definition at line 214 of file [Vector.h](#).

Here is the call graph for this function:

4.5.3.8 ~Vector()

```
template<typename T>
Vector< T >::~Vector () [inline]
```

Definition at line 217 of file [Vector.h](#).

4.5.4 Member Function Documentation

4.5.4.1 assign() [1/3]

```
template<typename T>
template<typename InputIt, typename = std::enable_if_t< std::is_base_of_v<std::input_iterator,
_tag, typename std::iterator_traits<InputIt>::iterator_category> >>
void Vector< T >::assign (
    InputIt first,
    InputIt last) [inline]
```

Definition at line 287 of file [Vector.h](#).

Here is the call graph for this function:

4.5.4.2 assign() [2/3]

```
template<typename T>
void Vector< T >::assign (
    size_type count,
    const T & value) [inline]
```

Definition at line 257 of file [Vector.h](#).

Here is the call graph for this function: Here is the caller graph for this function:

4.5.4.3 assign() [3/3]

```
template<typename T>
void Vector< T >::assign (
    std::initializer_list< T > ilist) [inline]
```

Definition at line 298 of file [Vector.h](#).

Here is the call graph for this function:

4.5.4.4 at() [1/2]

```
template<typename T>
reference Vector< T >::at (
    size_type pos) [inline]
```

Definition at line 310 of file [Vector.h](#).

Here is the caller graph for this function:

4.5.4.5 at() [2/2]

```
template<typename T>
const_reference Vector< T >::at (
    size_type pos) const [inline]
```

Definition at line 316 of file [Vector.h](#).

4.5.4.6 back() [1/2]

```
template<typename T>
reference Vector< T >::back () [inline]
```

Definition at line 344 of file [Vector.h](#).

Here is the caller graph for this function:

4.5.4.7 back() [2/2]

```
template<typename T>
const_reference Vector< T >::back () const [inline]
```

Definition at line 348 of file [Vector.h](#).

4.5.4.8 begin() [1/2]

```
template<typename T>
const_iterator Vector< T >::begin () const [inline], [noexcept]
```

Definition at line 366 of file [Vector.h](#).

4.5.4.9 begin() [2/2]

```
template<typename T>
iterator Vector< T >::begin () [inline], [noexcept]
```

Definition at line 363 of file [Vector.h](#).

Here is the caller graph for this function:

4.5.4.10 capacity()

```
template<typename T>
size_type Vector< T >::capacity () const [inline], [noexcept]
```

Definition at line 437 of file [Vector.h](#).

4.5.4.11 cbegin()

```
template<typename T>
const_iterator Vector< T >::cbegin () const [inline], [noexcept]
```

Definition at line 369 of file [Vector.h](#).

Here is the caller graph for this function:

4.5.4.12 cend()

```
template<typename T>
const_iterator Vector< T >::cend () const [inline], [noexcept]
```

Definition at line 379 of file [Vector.h](#).

Here is the caller graph for this function:

4.5.4.13 clear()

```
template<typename T>
void Vector< T >::clear () [inline], [noexcept]
```

Definition at line 461 of file [Vector.h](#).

Here is the call graph for this function: Here is the caller graph for this function:

4.5.4.14 crbegin()

```
template<typename T>
const_reverse_iterator Vector< T >::crbegin () const [inline], [noexcept]
```

Definition at line 389 of file [Vector.h](#).

Here is the call graph for this function:

4.5.4.15 crend()

```
template<typename T>
const_reverse_iterator Vector< T >::crend () const [inline], [noexcept]
```

Definition at line 399 of file [Vector.h](#).

Here is the call graph for this function:

4.5.4.16 data() [1/2]

```
template<typename T>
const T * Vector< T >::data () const [inline], [noexcept]
```

Definition at line 357 of file [Vector.h](#).

4.5.4.17 data() [2/2]

```
template<typename T>
T * Vector< T >::data () [inline], [noexcept]
```

Definition at line 354 of file [Vector.h](#).

4.5.4.18 `emplace()`

```
template<typename T>
template<typename... Args>
iterator Vector< T >::emplace (
    const_iterator pos,
    Args &&... args) [inline]
```

Definition at line 583 of file [Vector.h](#).

Here is the call graph for this function: Here is the caller graph for this function:

4.5.4.19 `emplace_back()`

```
template<typename T>
template<typename... Args>
reference Vector< T >::emplace_back (
    Args &&... args) [inline]
```

Definition at line 717 of file [Vector.h](#).

Here is the call graph for this function: Here is the caller graph for this function:

4.5.4.20 `empty()`

```
template<typename T>
bool Vector< T >::empty () const [inline], [noexcept]
```

Definition at line 406 of file [Vector.h](#).

Here is the caller graph for this function:

4.5.4.21 `end()` [1/2]

```
template<typename T>
const_iterator Vector< T >::end () const [inline], [noexcept]
```

Definition at line 376 of file [Vector.h](#).

4.5.4.22 `end()` [2/2]

```
template<typename T>
iterator Vector< T >::end () [inline], [noexcept]
```

Definition at line 373 of file [Vector.h](#).

Here is the caller graph for this function:

4.5.4.23 erase() [1/2]

```
template<typename T>
iterator Vector< T >::erase (
    const_iterator first,
    const_iterator last) [inline]
```

Definition at line 682 of file [Vector.h](#).

Here is the call graph for this function:

4.5.4.24 erase() [2/2]

```
template<typename T>
iterator Vector< T >::erase (
    const_iterator pos) [inline]
```

Definition at line 662 of file [Vector.h](#).

Here is the call graph for this function:

4.5.4.25 front() [1/2]

```
template<typename T>
reference Vector< T >::front () [inline]
```

Definition at line 334 of file [Vector.h](#).

4.5.4.26 front() [2/2]

```
template<typename T>
const_reference Vector< T >::front () const [inline]
```

Definition at line 338 of file [Vector.h](#).

4.5.4.27 getMemory_reallocated()

```
template<typename T>
int Vector< T >::getMemory_reallocated () [inline]
```

Definition at line 304 of file [Vector.h](#).

Here is the caller graph for this function:

4.5.4.28 insert() [1/3]

```
template<typename T>
iterator Vector< T >::insert (
    const_iterator pos,
    const T & value) [inline]
```

Definition at line 470 of file [Vector.h](#).

Here is the call graph for this function:

4.5.4.29 insert() [2/3]

```
template<typename T>
iterator Vector< T >::insert (
    const_iterator pos,
    size_type count,
    const T & value) [inline]
```

Definition at line 480 of file [Vector.h](#).

Here is the call graph for this function:

4.5.4.30 insert() [3/3]

```
template<typename T>
iterator Vector< T >::insert (
    const_iterator pos,
    T && value) [inline]
```

Definition at line 475 of file [Vector.h](#).

Here is the call graph for this function:

4.5.4.31 max_size()

```
template<typename T>
size_type Vector< T >::max_size () const [inline], [noexcept]
```

Definition at line 416 of file [Vector.h](#).

Here is the caller graph for this function:

4.5.4.32 operator=() [1/3]

```
template<typename T>
Vector & Vector< T >::operator= (
    const Vector< T > & other) [inline]
```

Definition at line 224 of file [Vector.h](#).

Here is the call graph for this function:

4.5.4.33 operator=() [2/3]

```
template<typename T>
Vector & Vector< T >::operator= (
    std::initializer_list< T > ilist) [inline]
```

Definition at line 251 of file [Vector.h](#).

Here is the call graph for this function:

4.5.4.34 operator=() [3/3]

```
template<typename T>
Vector & Vector< T >::operator= (
    Vector< T > && other) [inline], [noexcept]
```

Definition at line 233 of file [Vector.h](#).

Here is the call graph for this function:

4.5.4.35 operator[]() [1/2]

```
template<typename T>
reference Vector< T >::operator[] (
    size_type pos) [inline]
```

Definition at line 324 of file [Vector.h](#).

4.5.4.36 operator[]() [2/2]

```
template<typename T>
const_reference Vector< T >::operator[] (
    size_type pos) const [inline]
```

Definition at line 328 of file [Vector.h](#).

4.5.4.37 pop_back()

```
template<typename T>
void Vector< T >::pop_back () [inline]
```

Definition at line 729 of file [Vector.h](#).

Here is the call graph for this function: Here is the caller graph for this function:

4.5.4.38 push_back() [1/2]

```
template<typename T>
void Vector< T >::push_back (
    const T & value) [inline]
```

Definition at line 708 of file [Vector.h](#).

Here is the call graph for this function: Here is the caller graph for this function:

4.5.4.39 push_back() [2/2]

```
template<typename T>
void Vector< T >::push_back (
    T && value) [inline]
```

Definition at line 711 of file [Vector.h](#).

Here is the call graph for this function:

4.5.4.40 rbegin() [1/2]

```
template<typename T>
const_reverse_iterator Vector< T >::rbegin () const [inline], [noexcept]
```

Definition at line 386 of file [Vector.h](#).

Here is the call graph for this function:

4.5.4.41 rbegin() [2/2]

```
template<typename T>
reverse_iterator Vector< T >::rbegin () [inline], [noexcept]
```

Definition at line 383 of file [Vector.h](#).

Here is the call graph for this function:

4.5.4.42 rend() [1/2]

```
template<typename T>
const_reverse_iterator Vector< T >::rend () const [inline], [noexcept]
```

Definition at line 396 of file [Vector.h](#).

Here is the call graph for this function:

4.5.4.43 rend() [2/2]

```
template<typename T>
reverse_iterator Vector< T >::rend () [inline], [noexcept]
```

Definition at line 393 of file [Vector.h](#).

Here is the call graph for this function:

4.5.4.44 reserve()

```
template<typename T>
void Vector< T >::reserve (
    size_type new_cap) [inline]
```

Definition at line 425 of file [Vector.h](#).

Here is the call graph for this function: Here is the caller graph for this function:

4.5.4.45 resize() [1/2]

```
template<typename T>
void Vector< T >::resize (
    size_type count) [inline]
```

Definition at line 738 of file [Vector.h](#).

Here is the call graph for this function:

4.5.4.46 resize() [2/2]

```
template<typename T>
void Vector< T >::resize (
    size_type count,
    const value_type & value) [inline]
```

Definition at line 758 of file [Vector.h](#).

Here is the call graph for this function:

4.5.4.47 shrink_to_fit()

```
template<typename T>
void Vector< T >::shrink_to_fit () [inline]
```

Definition at line 442 of file [Vector.h](#).

Here is the call graph for this function: Here is the caller graph for this function:

4.5.4.48 size()

```
template<typename T>
size_type Vector< T >::size () const [inline], [noexcept]
```

Definition at line 411 of file [Vector.h](#).

Here is the caller graph for this function:

4.5.4.49 swap()

```
template<typename T>
void Vector< T >::swap (
    Vector< T > & other) [inline], [noexcept]
```

Definition at line 776 of file [Vector.h](#).

Here is the call graph for this function: Here is the caller graph for this function:

The documentation for this class was generated from the following file:

- [Vector.h](#)

Chapter 5

File Documentation

5.1 main.cpp File Reference

```
#include "template.h"
#include "program.cpp"
Include dependency graph for main.cpp:
```

Functions

- int [main](#) ()

5.1.1 Function Documentation

5.1.1.1 main()

```
int main ()
```

Definition at line [4](#) of file [main.cpp](#).

Here is the call graph for this function:

5.2 main.cpp

[Go to the documentation of this file.](#)

```
00001 #include "template.h"
00002 #include "program.cpp"
00003
00004 int main() {
00005     Vector <duom> grupe;
00006
00007     menu(grupe);
00008     return 0;
00009 }
```

5.3 program.cpp File Reference

```
#include "template.h"
```

Include dependency graph for program.cpp: This graph shows which files directly or indirectly include this file:

Functions

- void [menu](#) ([Vector](#)< [duom](#) > &grupe)
- void [read_from_console](#) ([Vector](#)< [duom](#) > &grupe)
- void [read_file](#) ([Vector](#)< [duom](#) > &grupe, string filename)
- void [read_names_from_console](#) ([Vector](#)< [duom](#) > &grupe)
- void [random_grades](#) ([Vector](#)< [duom](#) > &grupe, int m)
- void [random_names_grades](#) ([Vector](#)< [duom](#) > &grupe, int record_amount, int mark_amount)
- void [print_data_to_file](#) ([Vector](#)< [duom](#) > &grupe, int mark_amount, string filename)
- void [print_answers_to_file](#) ([Vector](#)< [duom](#) > &grupe, string filename)
- void [print_answers_console](#) ([Vector](#)< [duom](#) > &grupe)
- char [check_menu](#) ()
- void [vid_med_calc](#) ([Vector](#)< [duom](#) > &grupe)
- bool [compare](#) (const string a, const string b, string rule)
- void [sorting](#) ([Vector](#)< [duom](#) > &grupe, char rule)
- double [average](#) ([duom](#) given)
- double [median](#) ([duom](#) given)
- void [method_test](#) ([Vector](#)< [duom](#) > &grupe)
- void [class_write_test](#) ()

Variables

- [Vector](#)< string > [vardai](#)

5.3.1 Function Documentation

5.3.1.1 average()

```
double average (
    duom given)
```

Definition at line [482](#) of file [program.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.3.1.2 check_menu()

```
char check_menu ()
```

Definition at line [415](#) of file [program.cpp](#).

Here is the caller graph for this function:

5.3.1.3 class_write_test()

```
void class_write_test ()
```

Definition at line 558 of file [program.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.3.1.4 compare()

```
bool compare (  
    const string a,  
    const string b,  
    string rule)
```

Definition at line 458 of file [program.cpp](#).

5.3.1.5 median()

```
double median (  
    duom given)
```

Definition at line 499 of file [program.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.3.1.6 menu()

```
void menu (  
    Vector< duom > & grupe)
```

Definition at line 35 of file [program.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.3.1.7 method_test()

```
void method_test (  
    Vector< duom > & grupe)
```

Definition at line 525 of file [program.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.3.1.8 print_answers_console()

```
void print_answers_console (  
    Vector< duom > & grupe)
```

Definition at line 405 of file [program.cpp](#).

Here is the caller graph for this function:

5.3.1.9 print_answers_to_file()

```
void print_answers_to_file (
    Vector< duom > & grupe,
    string filename)
```

Definition at line 395 of file [program.cpp](#).

Here is the caller graph for this function:

5.3.1.10 print_data_to_file()

```
void print_data_to_file (
    Vector< duom > & grupe,
    int mark_amount,
    string filename)
```

Definition at line 362 of file [program.cpp](#).

Here is the caller graph for this function:

5.3.1.11 random_grades()

```
void random_grades (
    Vector< duom > & grupe,
    int m)
```

Definition at line 320 of file [program.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.3.1.12 random_names_grades()

```
void random_names_grades (
    Vector< duom > & grupe,
    int record_amount,
    int mark_amount)
```

Definition at line 338 of file [program.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.3.1.13 read_file()

```
void read_file (
    Vector< duom > & grupe,
    string filename)
```

Definition at line 251 of file [program.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.3.1.14 read_from_console()

```
void read_from_console (
    Vector< duom > & grupe)
```

Definition at line 195 of file [program.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.3.1.15 read_names_from_console()

```
void read_names_from_console (
    Vector< duom > & grupe)
```

Definition at line 304 of file [program.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.3.1.16 sorting()

```
void sorting (
    Vector< duom > & grupe,
    char rule)
```

Definition at line 465 of file [program.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.3.1.17 vid_med_calc()

```
void vid_med_calc (
    Vector< duom > & grupe)
```

Definition at line 433 of file [program.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.3.2 Variable Documentation

5.3.2.1 vardai

```
Vector<string> vardai
```

Definition at line 3 of file [program.cpp](#).

5.4 program.cpp

[Go to the documentation of this file.](#)

```

00001 #include "template.h"
00002
00003 Vector<string> vardai={
00004     "Tomas",
00005     "Andrius",
00006     "Daumantas",
00007     "Jonas",
00008     "Petras",
00009     "Kestas",
00010     "Paulius",
00011     "Juozas",
00012     "Rokas",
00013     "Adomas",
00014     "Amelija",
00015     "Motiejus",
00016     "Jonas",
00017     "Olivija",
00018     "Lukas",
00019     "Emilija",
00020     "Jokubas",
00021     "Adele",
00022     "Benas",
00023     "Ema",
00024     "Dominykas",
00025     "Liepa",
00026     "Nojus",
00027     "Ugne",
00028     "Matas",
00029     "Lukne",
00030     "Markas",
00031     "Barbora",
00032     "Augustas"
00033 };
00034
00035 void menu(Vector <duom> &grupe)
00036 {
00037     string input;
00038     char rule;
00039     do
00040     {
00041         cout<<"Jei duomenis norite ivesti patys (terminalas), spauskite '1'<<endl;
00042         cout<<"Jei duomenis norite ivesti patys (kursiokai.txt), spauskite '2'<<endl;
00043         cout<<"Jei norite sugeneruoti mokinio pazymius atsitiktinai, spauskite '3'<<endl;
00044         cout<<"Jei norite sugeneruoti mokinio pazymius ir vardus atsitiktinai (konsoleje), spauskite
00045         '4'<<endl;
00046         cout<<"Jei norite sugeneruoti mokinio pazymius ir vardus atsitiktinai (failuose), spauskite
00047         '5'<<endl;
00048         cout<<"Jei norite suruosiuoti mokinius pagal ju pazymius is failo, spauskite '6'<<endl;
00049         cout<<"Jei norite istestuoti metodu perdengima, spauskite '7'<<endl;
00050         cout<<"Jei norite istestuoti vector ir Vector klasiu spartos skirtuma, spauskite '8'<<endl;
00051         cout<<"Jei norite baigti darba, spauskite '9'<<endl;
00052         cin>>input;
00053         if(input.length()!=1)
00054             continue;
00055         rule=input.at(0);
00056         while(rule!='1' && rule!='2' && rule!='3' && rule!='4' && rule!='5' && rule!='6' && rule!='7' &&
00057         rule!='8');
00058         if(rule=='1')
00059         {
00060             read_from_console(grupe);
00061             vid_med_calc(grupe);
00062             sorting(grupe, '\0');
00063             print_answers_console(grupe);
00064         }
00065         else if (rule=='2')
00066         {
00067             grupė.reserve(1000);
00068             read_file(grupe, "kursiokai.dat");
00069             grupė.shrink_to_fit();
00070             vid_med_calc(grupe);
00071             sorting(grupe, '\0');
00072             print_answers_console(grupe);
00073         }
00074         else if (rule=='3')
00075         {
00076             grupė.reserve(1000);
00077             read_names_from_console(grupe);
00078             random_grades(grupe, grupė.size());
00079             grupė.shrink_to_fit();
00080             vid_med_calc(grupe);
00081             sorting(grupe, '\0');
00082             print_answers_console(grupe);
00083         }
00084     } while(rule!='9');
00085 }
```

```

00080     }
00081     else if (rule=='4')
00082     {
00083         random_names_grades(grupe, 10, 5);
00084         vid_med_calc(grupe);
00085         sorting(grupe, '\0');
00086         print_answers_console(grupe);
00087     }
00088     else if (rule=='5')
00089     {
00090         grupe.reserve(10000000);
00091         auto start = std::chrono::high_resolution_clock::now();
00092
00093         random_names_grades(grupe, 1000, 5);
00094         print_data_to_file(grupe, 5, "kursiokai_1000.dat");
00095         auto end = std::chrono::high_resolution_clock::now();
00096         cout << "1 failas sugeneruotas per: " << std::chrono::duration_cast<std::chrono::seconds>(end -
start).count() << " sec" << endl;
00097
00098         random_names_grades(grupe, 10000, 5);
00099         print_data_to_file(grupe, 5, "kursiokai_10000.dat");
00100         end = std::chrono::high_resolution_clock::now();
00101         cout << "2 failas sugeneruotas per: " <<
std::chrono::duration_cast<std::chrono::duration<double>(end - start).count() << " sec" << endl;
00102
00103         start = std::chrono::high_resolution_clock::now();
00104         random_names_grades(grupe, 100000, 5);
00105         print_data_to_file(grupe, 5, "kursiokai_100000.dat");
00106         end = std::chrono::high_resolution_clock::now();
00107         cout << "3 failas sugeneruotas per: " <<
std::chrono::duration_cast<std::chrono::duration<double>(end - start).count() << " sec" << endl;
00108
00109         start = std::chrono::high_resolution_clock::now();
00110         random_names_grades(grupe, 1000000, 5);
00111         print_data_to_file(grupe, 5, "kursiokai_1000000.dat");
00112         end = std::chrono::high_resolution_clock::now();
00113         cout << "4 failas sugeneruotas per: " <<
std::chrono::duration_cast<std::chrono::duration<double>(end - start).count() << " sec" << endl;
00114
00115         start = std::chrono::high_resolution_clock::now();
00116         random_names_grades(grupe, 10000000, 5);
00117         print_data_to_file(grupe, 5, "kursiokai_10000000.dat");
00118         end = std::chrono::high_resolution_clock::now();
00119         cout << "5 failas sugeneruotas per: " <<
std::chrono::duration_cast<std::chrono::duration<double>(end - start).count() << " sec" << endl;
00120     }
00121     else if (rule=='6')
00122     {
00123         string filename;
00124         cout<<"Iveskite norimo nuskaityti failo (.dat) pavadinima pvz. (vardai, duomenys, ...) "<<endl;
00125         cin>>filename;
00126         //
00127         // Start timing the entire process
00128         auto total_start = std::chrono::high_resolution_clock::now();
00129
00130         // Timing file reading
00131         auto start = std::chrono::high_resolution_clock::now();
00132         read_file(grupe, test_file_location + filename + ".dat");
00133         auto end = std::chrono::high_resolution_clock::now();
00134         auto read_time = std::chrono::duration_cast<std::chrono::milliseconds>(end - start).count();
00135
00136         // Timing user input
00137         start = std::chrono::high_resolution_clock::now();
00138         vid_med_calc(grupe);
00139         end = std::chrono::high_resolution_clock::now();
00140         auto excluded_time = std::chrono::duration_cast<std::chrono::milliseconds>(end -
start).count();
00141
00142         // Timing sorting
00143         start = std::chrono::high_resolution_clock::now();
00144         sorting(grupe, '3'); // Sorting by final grade
00145         end = std::chrono::high_resolution_clock::now();
00146         auto sort_time = std::chrono::duration_cast<std::chrono::milliseconds>(end - start).count();
00147
00148         // Splitting the vector into two
00149         start = std::chrono::high_resolution_clock::now();
00150
00151         Vector<duom> blogi;
00152         Vector<duom> geri;
00153         blogi.reserve(500);
00154         geri.reserve(500);
00155
00156         for(auto &i:grupe)
00157         {
00158             if(i.getMark()<5)
00159                 blogi.push_back(i);
00160             else

```

```

00161         geri.push_back(i);
00162     }
00163     grupe.clear();
00164
00165     end = std::chrono::high_resolution_clock::now();
00166     auto splitting_time = std::chrono::duration_cast<std::chrono::milliseconds>(end -
start).count();
00167
00168     start = std::chrono::high_resolution_clock::now();
00169     print_answers_to_file(grupe, test_file_location + "../output_file/" + filename +
"kietekai.dat");
00170     print_answers_to_file(blogi, test_file_location + "../output_file/" + filename + +
"_vargsiukai.dat");
00171     end = std::chrono::high_resolution_clock::now();
00172     auto write_time = std::chrono::duration_cast<std::chrono::milliseconds>(end - start).count();
00173
00174     //Calculate and display total time
00175     auto total_end = std::chrono::high_resolution_clock::now();
00176     auto raw_time = std::chrono::duration_cast<std::chrono::milliseconds>(total_end -
total_start).count();
00177     auto total_time = raw_time - excluded_time;
00178
00179     // Display summary of times
00180     cout << "\n\nSantrauka:" << endl;
00181     cout << "Skaitymas: " << float(read_time) / 1000 << " s " << endl;
00182     cout << "Rusiavimas: " << float(sort_time) / 1000 << " s " << endl;
00183     cout << "Skaitymas: " << float(splitting_time) / 1000 << " s " << endl;
00184     cout << "Rasydas: " << float(write_time) / 1000 << " s " << endl;
00185     cout << "Is viso: " << float(total_time) / 1000 << " s " << endl;
00186 }
00187 else if (rule=='7')
00188     method_test(grupe);
00189 else if (rule=='8')
00190     class_write_test();
00191 else if (rule=='9')
00192     cout<<"Darbas baigtas"<<endl;
00193     cout<<"Atmintis persikirstyta: " << grupe.getMemory_reallocated() << " karta(-us)" <<endl;
00194 }
00195 void read_from_console(Vector <duom> &grupe)
00196 {
00197     bool con=1;
00198     char check;
00199     temp local;
00200     while(con)
00201     {
00202         cout<<"Iveskite varda ir pavarde"<<endl;
00203         cin>>local.var>>local.pav;
00204         cout<<"Iveskite pazymius (paskutinis pazymys is egzamino) (iveskite -1 norint uzbaigti)"<<endl;
00205         int temp;
00206         while(true)
00207         {
00208             try
00209             {
00210                 cin>>temp;
00211                 if (cin.fail() || temp < -1 || temp > 10)
00212                 {
00213                     cin.clear();
00214                     throw invalid_argument("Netinkama ivestis");
00215                 }
00216             }
00217             catch(const exception& e)
00218             {
00219                 cerr << e.what() << '\n';
00220                 continue;
00221             }
00222             if (temp == -1)
00223                 break;
00224             else
00225                 local.pazymiai.push_back(temp);
00226         }
00227         try
00228         {
00229             if(local.pazymiai.size()==0)
00230                 throw invalid_argument("Neivesti pazymiai");
00231         }
00232         catch(const std::exception& e)
00233         {
00234             std::cerr << e.what() << '\n';
00235             terminate();
00236         }
00237         local.exam=local.pazymiai.back();
00238         local.pazymiai.pop_back();
00239         //
00240         duom to_push(local);
00241         grupe.push_back(to_push);
00242         //
00243         local.pazymiai.clear();

```

```

00244         local.exam=0;
00245         cout<<"Noredami uzbaigti ivedima iveskite 0"<<endl;
00246         cin>>check;
00247         if(check=='0')
00248             con=0;
00249     }
00250 }
00251 void read_file(Vector <duom> &grupe, string filename)
00252 {
00253     ifstream in(filename);
00254     try
00255     {
00256         if(!in)
00257         {
00258             throw invalid_argument("Failas nerastas");
00259         }
00260     }
00261     catch(const std::exception& e)
00262     {
00263         std::cerr << e.what() << '\n';
00264         terminate();
00265     }
00266
00267     in.seekg(0, ios::end); //pointeris i gala
00268     streamsize size = in.tellg(); //nustatomas failo dydis
00269     in.seekg(0, ios::beg); //pointeris i pradzia
00270
00271     try
00272     {
00273         if(size>100000000)
00274             grupe.reserve(10000000);
00275         else if(size>10000000)
00276             grupe.reserve(1000000);
00277         else if(size>1000000)
00278             grupe.reserve(100000);
00279         else if(size>100000)
00280             grupe.reserve(10000);
00281         else
00282             grupe.reserve(1000);
00283     }
00284     catch(const std::exception& e)
00285     {
00286         std::cerr << "Nepavyko rezervuoti atminties" << '\n';
00287     }
00288
00289     string header_line;
00290     std::getline(in, header_line);
00291     //
00292     duom laik;
00293     try
00294     {
00295         while(in>>laik)
00296         {
00297             grupe.push_back(std::move(laik));
00298         }
00299     }
00300     catch(const std::exception& e){}
00301     grupe.shrink_to_fit();
00302 }
00303
00304 void read_names_from_console(Vector <duom> &grupe)
00305 {
00306     bool con=1;
00307     temp local;
00308     while(con)
00309     {
00310         cout<<"Iveskite varda ir pavarde"<<endl;
00311         cin>>local.var>>local.pav;
00312         //
00313         duom to_push(local);
00314         grupe.push_back(std::move(to_push));
00315         //
00316         if(check_menu()=='F')
00317             con=0;
00318     }
00319 }
00320 void random_grades(Vector <duom> &grupe, int m)
00321 {
00322     temp laik;
00323     random_device rd;
00324     mt19937 mt(rd());
00325     uniform_int_distribution<int> dist(1, 10);
00326     uniform_int_distribution<int> dist2(5, 20);
00327     int n = dist2(mt);
00328     for(int j=0; j<m; j++)
00329     {
00330         for(int i=0; i<n; i++)

```

```

00331         laik.pazymiai.push_back(dist(mt));
00332         laik.exam=(dist(mt));
00333         grupe[j].setPazymiai(laik.pazymiai);
00334         grupe[j].setExam(laik.exam);
00335         laik.pazymiai.clear();
00336     }
00337 }
00338 void random_names_grades(Vector <duom> &grupe, int record_amount, int mark_amount)
00339 {
00340     temp laik;
00341     random_device rd;
00342     mt19937 mt(rd());
00343     uniform_int_distribution<int> dist(1, 10);
00344     uniform_int_distribution<int> dist3(0, 27); //random vardu generavimas
00345     uniform_int_distribution<int> dist4(65, 86); //ASCII values for uppercase letters
00346     grupe.clear();
00347     for(int j=0; j<record_amount; j++)
00348     {
00349         laik.var=vardai.at(dist3(mt));
00350         laik.pav=dist4(mt);
00351         laik.pav+=".";
00352         for(int i=0; i<mark_amount; i++)
00353             laik.pazymiai.push_back(dist(mt));
00354         laik.exam=(dist(mt));
00355         //
00356         duom to_push(laik);
00357         grupe.push_back(to_push);
00358         //
00359         laik.pazymiai.clear();
00360     }
00361 }
00362 void print_data_to_file(Vector <duom> &grupe, int mark_amount, string filename)
00363 {
00364     ofstream out(filename);
00365     stringstream ss;
00366     const int ss_size = 10000; //stringstream max eiluciu skaicius
00367     int count = 0;
00368     //
00369     ss << left << fixed << setw(20) << "Vardas"<<setw(20)<<"Pavarde";
00370     for(int i=0; i<mark_amount; i++)
00371     {
00372         ss << setw(7) << "ND" + to_string(i+1);
00373     }
00374     ss << setw(10) << "Egzaminas" << endl;
00375     //
00376     for(auto i:grupe)
00377     {
00378         ss << setw(20) << i.getVar() << setw(20) << i.getPav();
00379         for(auto j:i.getPazymiai())
00380             ss << setw(7) << j;
00381         ss << setw(10) << i.getExam() << endl;
00382         count++; //eiluciu skaicius ss stringstreame
00383         if(count==ss_size)
00384         {
00385             out<<ss.str();
00386             ss.str("");
00387             count=0;
00388             ss.clear();
00389         }
00390     }
00391     if(count!=0) //likusiu eiluciu isvedimas
00392         out<<ss.str();
00393     out.close();
00394 }
00395 void print_answers_to_file(Vector <duom> &grupe, string filename)
00396 {
00397     ofstream out(filename);
00398     out << left << fixed << setw(20) << "Vardas"<<setw(20)<<"Pavarde"
00399     << fixed << setprecision(2) << setw(20) << "Galutinis" << endl;
00400     //
00401     for(auto i:grupe)
00402         out<<i;
00403     out.close();
00404 }
00405 void print_answers_console(Vector <duom> &grupe)
00406 {
00407     cout << left << fixed << setw(20) << "Vardas"<<setw(20)<<"Pavarde"
00408     << fixed << setprecision(2) << setw(20) << "Galutinis" << endl;
00409     cout << "-----" << endl;
00410     //
00411     for(auto i:grupe)
00412         cout<<i;
00413 }
00414 //
00415 char check_menu()
00416 {
00417     string input;

```



```

00418     char check;
00419     do
00420     {
00421         cout<<"Noredami uzbaigti ivedima iveskite 'F'"<<endl;
00422         cout<<"Noredami testi ivedima iveskite 'T'"<<endl;
00423         getline(cin, input);
00424         if(input.size()>1 || input.size()==0)
00425         {
00426             cout<<"Netinkama ivistis"<<endl;
00427             continue;
00428         }
00429         check=toupper(input[0]);
00430     } while (check!='F' && check!='T');
00431     return check;
00432 }
00433 void vid_med_calc(Vector <duom> &grupe)
00434 {
00435     char rule;
00436     do
00437     {
00438         cout<<"Noredami skaiciuoti naudojant vidurki parasykite 'v'"<<endl;
00439         cout<<"Noredami skaiciuoti naudojant mediana parasykite 'm'"<<endl;
00440         cin>>rule;
00441         rule=tolower(rule);
00442     } while(rule!='v' && rule!='m');
00443     if(rule=='v')
00444     {
00445         for(auto &i:grupe)
00446             i.setVid_med(average(i));
00447     }
00448     else
00449     {
00450         for(auto &i:grupe)
00451             i.setVid_med(median(i));
00452     }
00453     for(auto &i:grupe)
00454     {
00455         i.setMark(0.4*i.getVid_med()+0.6*i.getExam());
00456     }
00457 }
00458 bool compare(const string a, const string b, string rule)
00459 {
00460     if(a.rfind(rule, 0)==0 && b.rfind(rule, 0)==0)
00461         return stoi(a.substr(rule.length()))<stoi(b.substr(rule.length()));
00462     else
00463         return a<b;
00464 }
00465 void sorting(Vector <duom> &grupe, char rule)
00466 {
00467     while(rule!='1' && rule!='2' && rule!='3')
00468     {
00469         cout<<"Noredami surusiuoti pagal varda, spauskite '1'"<<endl;
00470         cout<<"Noredami surusiuoti pagal pavarde, spauskite '2'"<<endl;
00471         cout<<"Noredami surusiuoti pagal galutini bala, spauskite '3'"<<endl;
00472         cin>>rule;
00473         rule=tolower(rule);
00474     }
00475     if(rule=='1')
00476         std::stable_sort( grupe.begin(), grupe.end(), [](const duom& a, const duom& b){return
compare(a.getVar(), b.getVar(), "Vardas");});
00477     else if(rule=='2')
00478         std::stable_sort( grupe.begin(), grupe.end(), [](const duom& a, const duom& b){return
compare(a.getPav(), b.getPav(), "Vardas");});
00479     else if(rule=='3')
00480         std::stable_sort( grupe.begin(), grupe.end(), [](const duom& a, const duom& b){return
a.getMark() > b.getMark();});
00481 }
00482 double average(duom given)
00483 {
00484     try
00485     {
00486         if(given.getPazymiai().empty())
00487             throw invalid_argument("Truksta pazymiu");
00488     }
00489     catch(const std::exception& e)
00490     {
00491         std::cerr << e.what() << '\n';
00492         terminate();
00493     }
00494     double sum=0.0;
00495     for(auto i:given.getPazymiai())
00496         sum+=i;
00497     return sum/given.getPazymiai().size();
00498 }
00499 double median(duom given)
00500 {
00501     try

```

```

00502     {
00503         if(given.getPazymiai().size()==0)
00504             throw invalid_argument("Truksta pazymiu");
00505     }
00506     catch(const std::exception& e)
00507     {
00508         std::cerr << e.what() << '\n';
00509         terminate();
00510     }
00511     if(given.getPazymiai().size()%2==1)
00512     {
00513         return given.getPazymiai()[((given.getPazymiai().size()/2)+1)-1];
00514     }
00515     else
00516     {
00517         double ats=0.0;
00518         ats=given.getPazymiai()[given.getPazymiai().size()/2-1];
00519         ats+=given.getPazymiai()[given.getPazymiai().size()/2+1-1];
00520         ats/=2;
00521         return ats;
00522     }
00523 }
00524 }
00525 void method_test(Vector <duom> &grupe)
00526 {
00527     read_file(grupe, test_file_location + "kursiokai_1000" + ".dat");
00528     duom test_case = grupė.at(10);
00529     //
00530     duom copy_constr(test_case);
00531     if(copy_constr==test_case)
00532         cout<<"copy constr succesful"<<endl;
00533     else
00534         cout<<"copy constr unsuccesful"<<endl;
00535     duom copy_method;
00536     copy_method=test_case;
00537     if(copy_method==test_case)
00538         cout<<"copy method succesful"<<endl;
00539     else
00540         cout<<"copy method unsuccesful"<<endl;
00541     duom move_constr(std::move(test_case));
00542     if(move_constr==grupe.at(10) && test_case.isClear())
00543         cout<<"move constr succesful"<<endl;
00544     else
00545         cout<<"move constr unsuccesful"<<endl;
00546     test_case = grupė.at(10);
00547     duom move_method;
00548     move_method=std::move(test_case);
00549     if(move_method==grupe.at(10) && test_case.isClear())
00550         cout<<"move method succesful"<<endl;
00551     else
00552         cout<<"move method unsuccesful"<<endl;
00553 }
00554 void class_write_test()
00555 {
00556     vector <duom> stl_vector;
00557     Vector <duom> new_vector;
00558     duom obj;
00559
00560     auto start = std::chrono::high_resolution_clock::now();
00561     auto end = std::chrono::high_resolution_clock::now();
00562     auto total = std::chrono::duration_cast<std::chrono::seconds>(end - start).count();
00563
00564     for(int i=1000; i<=100000000; i*=10)
00565     {
00566         start = std::chrono::high_resolution_clock::now();
00567         for(int j=0; j<i; j++)
00568             stl_vector.push_back(obj);
00569         end = std::chrono::high_resolution_clock::now();
00570         total = std::chrono::duration_cast<std::chrono::seconds>(end - start).count();
00571         cout<<"Laikas skirtas uzpildyti stl::vector tipo struktura" << i << " objektu: " << total << "
00572         s"<<endl;
00573         //
00574         stl_vector.clear();
00575         stl_vector.shrink_to_fit();
00576         //
00577         start = std::chrono::high_resolution_clock::now();
00578         for(int j=0; j<i; j++)
00579             new_vector.push_back(obj);
00580         end = std::chrono::high_resolution_clock::now();
00581         total = std::chrono::duration_cast<std::chrono::seconds>(end - start).count();
00582         cout<<"Laikas skirtas uzpildyti Vector tipo struktura" << i << " objektu: " << total << " s"<<endl;
00583         cout<<"///"<<endl;
00584         //
00585         new_vector.clear();
00586         new_vector.shrink_to_fit();
00587     }
00588 }
00589 }
00590 }
00591 }

```

5.5 std.h File Reference

This graph shows which files directly or indirectly include this file:

5.6 std.h

[Go to the documentation of this file.](#)

```

00001 #ifndef STD_H_INCLUDED
00002 #define STD_H_INCLUDED
00003
00004 using std::cout;
00005 using std::cin;
00006 using std::string;
00007 using std::endl;
00008 using std::vector;
00009 using std::ifstream;
00010 using std::setprecision;
00011 using std::fixed;
00012 using std::stringstream;
00013 using std::terminate;
00014 using std::sort;
00015 using std::setw;
00016 using std::left;
00017 using std::stringstream;
00018 using std::invalid_argument;
00019 using std::exception;
00020 using std::cerr;
00021 using std::stoi;
00022 using std::tolower;
00023 using std::numeric_limits;
00024 using std::streamsize;
00025 using std::random_device;
00026 using std::mt19937;
00027 using std::uniform_int_distribution;
00028 using std::ofstream;
00029 using std::to_string;
00030 using std::ios;
00031 using namespace std::chrono;
00032
00033 #endif

```

5.7 Studentai_tests.cpp File Reference

```

#include "template.h"
#include <gtest/gtest.h>
Include dependency graph for Studentai_tests.cpp:

```

5.8 Studentai_tests.cpp

[Go to the documentation of this file.](#)

```

00001 #include "template.h"
00002 #include <gtest/gtest.h>
00003
00004 class StudentaiTest : public ::testing::Test {
00005 protected:
00006     void SetUp() override {
00007         testNd = {8, 9, 7, 10, 6};
00008     }
00009     Vector<int> testNd;
00010
00011     duom createSampleStudent() {
00012         temp input;
00013         input.pazymiai = testNd;
00014         duom test(input);
00015         return test;

```

```

00016     }
00017 };
00018 TEST_F(StudentaiTest, Random_grade_check)
00019 {
00020     Vector<duom> test_vector;
00021
00022     // Generate random names and grades for 10 students, each with 5 grades
00023     random_names_grades(test_vector, 10, 5);
00024
00025     // Check that the vector contains 10 students
00026     EXPECT_EQ(test_vector.size(), 10);
00027
00028     for (auto& student : test_vector)
00029     {
00030         // Check that each student has 5 grades
00031         EXPECT_EQ(student.getPazymiai().size(), 5);
00032
00033         for (auto& mark : student.getPazymiai())
00034         {
00035             // Check that each grade is between 1 and 10
00036             EXPECT_GE(mark, 1);
00037             EXPECT_LE(mark, 10);
00038         }
00039     }
00040 }
00041 TEST_F(StudentaiTest, Handles_zero_students)
00042 {
00043     Vector<duom> test_vector;
00044
00045     // Generate random names and grades for 0 students
00046     random_names_grades(test_vector, 0, 5);
00047
00048     // Check that the vector is empty
00049     EXPECT_EQ(test_vector.size(), 0);
00050 }
00051 TEST_F(StudentaiTest, Handles_zero_grades)
00052 {
00053     Vector<duom> test_vector;
00054
00055     // Generate random names and grades for 10 students, each with 0 grades
00056     random_names_grades(test_vector, 10, 0);
00057
00058     // Check that the vector contains 10 students
00059     EXPECT_EQ(test_vector.size(), 10);
00060
00061     for (auto& student : test_vector)
00062     {
00063         // Check that each student has 0 grades
00064         EXPECT_EQ(student.getPazymiai().size(), 0);
00065     }
00066 }
00067 TEST_F(StudentaiTest, Handles_data_correctly)
00068 {
00069     Vector<duom> test_vector;
00070     bool rule;
00071
00072     read_file(test_vector, test_file_location + "kursiokai_1000" + ".dat");
00073     //
00074     duom test_case = test_vector.at(0);
00075     //
00076     duom copy_constr = test_case;
00077     if(copy_constr==test_case)
00078         rule=true;
00079     else
00080         rule=false;
00081     EXPECT_EQ(rule, true);
00082     //
00083     duom copy_method;
00084     copy_method=test_case;
00085     if(copy_method==test_case)
00086         rule=true;
00087     else
00088         rule=false;
00089     EXPECT_EQ(rule, true);
00090     //
00091     duom move_constr = std::move(test_case);
00092     if(move_constr==test_case && move_constr==test_vector.at(0))
00093         rule=false;
00094     else
00095         rule=true;
00096     EXPECT_EQ(rule, true);
00097     //
00098     test_case = test_vector.at(0);
00099     duom move_method;
00100     move_method=std::move(test_case);
00101     if(move_method==test_case && move_method==test_vector.at(0))
00102         rule=false;

```

```

00103     else
00104         rule=true;
00105     EXPECT_EQ(rule, true);
00106 }

```

5.9 template.h File Reference

```

#include <iostream>
#include <fstream>
#include <iomanip>
#include <string>
#include <sstream>
#include <cstdlib>
#include <ctime>
#include <algorithm>
#include <random>
#include <chrono>
#include <execution>
#include "std.h"
#include <vector>
#include "Vector.h"

```

Include dependency graph for template.h: This graph shows which files directly or indirectly include this file:

Classes

- struct [temp](#)
- class [human](#)
- class [duom](#)

Functions

- void [menu](#) ([Vector](#)< [duom](#) > &grupe)
- void [read_from_console](#) ([Vector](#)< [duom](#) > &grupe)
- void [read_file](#) ([Vector](#)< [duom](#) > &grupe, string filename)
- void [read_names_from_console](#) ([Vector](#)< [duom](#) > &grupe)
- void [random_grades](#) ([Vector](#)< [duom](#) > &grupe, int m)
- void [random_names_grades](#) ([Vector](#)< [duom](#) > &grupe, int record_amount, int mark_amount)
- void [print_data_to_file](#) ([Vector](#)< [duom](#) > &grupe, int mark_amount, string filename)
- void [print_answers_to_file](#) ([Vector](#)< [duom](#) > &grupe, string filename)
- void [print_answers_console](#) ([Vector](#)< [duom](#) > &grupe)
- char [check_menu](#) ()
- void [vid_med_calc](#) ([Vector](#)< [duom](#) > &grupe)
- bool [compare](#) (const string a, const string b, string rule)
- void [sorting](#) ([Vector](#)< [duom](#) > &grupe, char rule)
- double [average](#) ([duom](#) given)
- double [median](#) ([duom](#) given)
- void [method_test](#) ([Vector](#)< [duom](#) > &grupe)
- void [class_write_test](#) ()

Variables

- const string [test_file_location](#) = TEST_FILE_LOCATION

5.9.1 Function Documentation

5.9.1.1 average()

```
double average (  
    duom given)
```

Definition at line 482 of file [program.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.1.2 check_menu()

```
char check_menu ()
```

Definition at line 415 of file [program.cpp](#).

Here is the caller graph for this function:

5.9.1.3 class_write_test()

```
void class_write_test ()
```

Definition at line 558 of file [program.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.1.4 compare()

```
bool compare (  
    const string a,  
    const string b,  
    string rule)
```

Definition at line 458 of file [program.cpp](#).

5.9.1.5 median()

```
double median (  
    duom given)
```

Definition at line 499 of file [program.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.1.6 menu()

```
void menu (  
    Vector< duom > & grupe)
```

Definition at line 35 of file [program.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.1.7 method_test()

```
void method_test (  
    Vector< duom > & grupe)
```

Definition at line 525 of file [program.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.1.8 print_answers_console()

```
void print_answers_console (  
    Vector< duom > & grupe)
```

Definition at line 405 of file [program.cpp](#).

Here is the caller graph for this function:

5.9.1.9 print_answers_to_file()

```
void print_answers_to_file (  
    Vector< duom > & grupe,  
    string filename)
```

Definition at line 395 of file [program.cpp](#).

Here is the caller graph for this function:

5.9.1.10 print_data_to_file()

```
void print_data_to_file (  
    Vector< duom > & grupe,  
    int mark_amount,  
    string filename)
```

Definition at line 362 of file [program.cpp](#).

Here is the caller graph for this function:

5.9.1.11 random_grades()

```
void random_grades (
    Vector< duom > & grupe,
    int m)
```

Definition at line 320 of file [program.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.1.12 random_names_grades()

```
void random_names_grades (
    Vector< duom > & grupe,
    int record_amount,
    int mark_amount)
```

Definition at line 338 of file [program.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.1.13 read_file()

```
void read_file (
    Vector< duom > & grupe,
    string filename)
```

Definition at line 251 of file [program.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.1.14 read_from_console()

```
void read_from_console (
    Vector< duom > & grupe)
```

Definition at line 195 of file [program.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.1.15 read_names_from_console()

```
void read_names_from_console (
    Vector< duom > & grupe)
```

Definition at line 304 of file [program.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.1.16 sorting()

```
void sorting (
    Vector< duom > & grupe,
    char rule)
```

Definition at line 465 of file [program.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.1.17 vid_med_calc()

```
void vid_med_calc (
    Vector< duom > & grupe)
```

Definition at line 433 of file [program.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.2 Variable Documentation

5.9.2.1 test_file_location

```
const string test_file_location = TEST_FILE_LOCATION
```

Definition at line 256 of file [template.h](#).

5.10 template.h

[Go to the documentation of this file.](#)

```
00001 #ifndef TEMPLATE_H_INCLUDED
00002 #define TEMPLATE_H_INCLUDED
00003
00004 #include <iostream>
00005 #include <fstream>
00006 #include <iomanip>
00007 #include <string>
00008 #include <sstream>
00009 #include <cstdlib>
00010 #include <ctime>
00011 #include <algorithm>
00012 #include <random>
00013 #include <string>
00014 #include <chrono>
00015 #include <execution>
00016 #include "std.h"
00017 #include <vector>
00018 #include "Vector.h"
00019
00020 struct temp
00021 {
00022     string var="test";
00023     string pav="test";
00024     Vector<int> pazymiai;
00025     int exam=0;
00026     double vid_med=0;
00027     double mark;
00028 };
00029
00030 class human
00031 {
```

```

00032     public:
00033         virtual void abstractClassFunction() = 0;
00035     protected:
00036         string var = "";
00037         string pav = "";
00038 };
00039
00040
00041 class duom : private human
00042 {
00043     public:
00044         void abstractClassFunction() override{};
00045
00046     private:
00047         Vector<int> pazymiai;
00048         int exam = 0;
00049         double vid_med = 0.0;
00050         double mark = 0.0;
00051     public:
00052         duom(temp &a)
00053         {
00054             this->var=a.var;
00055             this->pav=a.pav;
00056             this->pazymiai=a.pazymiai;
00057             this->exam=a.exam;
00058             this->vid_med=a.vid_med;
00059             this->mark=a.mark;
00060         }
00061         duom(string var, string pav)
00062         {
00063             this->var=var;
00064             this->pav=pav;
00065         }
00066         duom(string var, string pav, Vector<int> &pazymiai, int exam)
00067         {
00068             this->var=var;
00069             this->pav=pav;
00070             this->pazymiai=pazymiai;
00071             this->exam=exam;
00072         }
00073         duom()
00074         {}
00075         ~duom()
00076         {
00077             var.clear();
00078             pav.clear();
00079             pazymiai.clear();
00080             exam=0;
00081             vid_med=0;
00082             mark=0;
00083         }
00085         duom(const duom &to_copy)
00086         {
00087             this->var=to_copy.var;
00088             this->pav=to_copy.pav;
00089             this->pazymiai=to_copy.pazymiai;
00090             this->exam=to_copy.exam;
00091             this->vid_med=to_copy.vid_med;
00092             this->mark=to_copy.mark;
00093         }
00094
00095         duom(duom &&to_move) noexcept
00096         {
00097             this->var=std::move(to_move.var);
00098             this->pav=std::move(to_move.pav);
00099             this->pazymiai=std::move(to_move.pazymiai);
00100             this->exam=to_move.exam;
00101             to_move.exam=0;
00102             this->vid_med=to_move.vid_med;
00103             to_move.vid_med=0;
00104             this->mark=to_move.mark;
00105             //to_move.mark=0;
00106         }
00107         //
00109         duom& operator=(const duom &to_copy)
00110         {
00111             if(this == &to_copy) return *this;
00112             //
00113             this->var=to_copy.var;
00114             this->pav=to_copy.pav;
00115             this->pazymiai=to_copy.pazymiai;
00116             this->exam=to_copy.exam;
00117             this->vid_med=to_copy.vid_med;
00118             this->mark=to_copy.mark;
00119             return *this;
00120         }
00121         duom& operator=(duom &&to_move) noexcept

```

```

00122     {
00123         if(this == &to_move) return *this;
00124         //
00125         this->var=std::move(to_move.var);
00126         this->pav=std::move(to_move.pav);
00127         this->pazymiai=std::move(to_move.pazymiai);
00128         this->exam=to_move.exam;
00129         to_move.exam=0;
00130         this->vid_med=to_move.vid_med;
00131         to_move.vid_med=0;
00132         this->mark=to_move.mark;
00133         to_move.mark=0;
00134         return *this;
00135     }
00136 }
00137 bool isClear() noexcept
00138 {
00139     if(this->var==" " && this->pav==" " && this->pazymiai.empty() && this->exam==0 &&
this->vid_med==0 && this->mark==0)
00140         return true;
00141     return false;
00142 }
00143 bool operator==(duom &&to_compare) noexcept
00144 {
00145     if(this->var==to_compare.var && this->pav==to_compare.pav &&
this->pazymiai==to_compare.pazymiai && this->exam==to_compare.exam &&
this->vid_med==to_compare.vid_med
00146     && this->mark==to_compare.mark)
00147         return true;
00148     else
00149         return false;
00150 }
00151 friend bool operator==(const duom &a, const duom &b) noexcept
00152 {
00153     if(a.var==b.var && a.pav==b.pav && a.pazymiai==b.pazymiai && a.exam==b.exam &&
a.vid_med==b.vid_med
00154     && a.mark==b.mark)
00155         return true;
00156     return false;
00157 }
00159 friend std::ifstream& operator>>(std::ifstream& in, duom& student)
00160 {
00161     string eil;
00162     try
00163     {
00164         std::getline(in, eil);
00165     }
00166     catch(const std::exception& e)
00167     {
00168         std::cerr << e.what() << '\n';
00169     }
00170
00171     stringstream line(eil);
00172     line>>student.var>>student.pav;
00173
00174     double grade;
00175     student.pazymiai.clear();
00176     while(line>>grade)
00177         student.pazymiai.push_back(grade);
00178
00179     if(student.pazymiai.size()==0)
00180         throw invalid_argument("Truksta pazymiu");
00181
00182     student.exam=student.pazymiai.back();
00183     student.pazymiai.pop_back();
00184
00185     return in;
00186 }
00187 friend std::ostream& operator<<(std::ostream& out, duom& student)
00188 {
00189     return out << left << fixed << setprecision(2) << setw(20) << student.var << " " << setw(20) <<
student.pav << " " << setw(20) << student.mark << endl;
00190 }
00192 string getVar() const
00193 {
00194     return var;
00195 }
00196 string getPav() const
00197 {
00198     return pav;
00199 }
00200 int getPazymiai_at(int i) const
00201 {
00202     return pazymiai[i];
00203 }
00204 Vector<int> getPazymiai() const
00205 {
00206     return pazymiai;

```

```

00207     }
00208     int getExam() const
00209     {
00210         return exam;
00211     }
00212     double getVid_med() const
00213     {
00214         return vid_med;
00215     }
00216     double getMark() const
00217     {
00218         return mark;
00219     }
00221     void setPazymiai(Vector<int> &pazymiai)
00222     {
00223         this->pazymiai=pazymiai;
00224     }
00225     void addPazymiai(int grade)
00226     {
00227         if(grade>=1 && grade <=10)
00228             pazymiai.push_back(grade);
00229         else
00230             throw("Neteisingas pazymys");
00231     }
00232     void setExam(int exam)
00233     {
00234         if(exam>=1 && exam <=10)
00235             this->exam = exam;
00236         else
00237             throw("Neteisingas pazymys");
00238     }
00239     void setVid_med(double vid_med)
00240     {
00241         if(vid_med>=1 && vid_med <=10)
00242             this->vid_med=vid_med;
00243         else
00244             throw("Neteisingas pazymys");
00245     }
00246     void setMark(double mark)
00247     {
00248         if(mark>=1 && mark <=10)
00249             this->mark=mark;
00250         else
00251             throw("Neteisingas pazymys");
00252     }
00254 };
00255
00256 const string test_file_location = TEST_FILE_LOCATION; //CMake version
00257 //const string test_file_location = "../test_files/"; //Manual compilation version (debug)
00258
00259
00260 void menu(Vector <duom> &grupe);
00261 void read_from_console(Vector <duom> &grupe);
00262 void read_file(Vector <duom> &grupe, string filename);
00263 void read_names_from_console(Vector <duom> &grupe);
00264 void random_grades(Vector <duom> &grupe, int m);
00265 void random_names_grades(Vector <duom> &grupe, int record_amount, int mark_amount);
00266 void print_data_to_file(Vector <duom> &grupe, int mark_amount, string filename);
00267 void print_answers_to_file(Vector <duom> &grupe, string filename);
00268 void print_answers_console(Vector <duom> &grupe);
00269 //
00270 char check_menu();
00271 void vid_med_calc(Vector <duom> &grupe);
00272 bool compare(const string a, const string b, string rule);
00273 void sorting(Vector <duom> &grupe, char rule);
00274 double average(duom given);
00275 double median(duom given);
00276 //
00277 void method_test(Vector <duom> &grupe);
00278 void class_write_test();
00279
00280 #endif
00281
00282

```

5.11 Vector.h File Reference

```

#include <cstdint>
#include <stdexcept>
#include <iterator>

```

```
#include <memory>
#include <utility>
#include <initializer_list>
#include <algorithm>
#include <limits>
#include <type_traits>
```

Include dependency graph for Vector.h: This graph shows which files directly or indirectly include this file:

Classes

- class [Vector< T >](#)

Functions

- template<typename T>
void [destroy_range](#) (T *first, T *last)
- template<typename T>
void [swap](#) ([Vector](#)< T > &lhs, [Vector](#)< T > &rhs) noexcept
- template<typename T>
bool [operator==](#) (const [Vector](#)< T > &lhs, const [Vector](#)< T > &rhs)
- template<typename T>
bool [operator!=](#) (const [Vector](#)< T > &lhs, const [Vector](#)< T > &rhs)
- template<typename T>
bool [operator<](#) (const [Vector](#)< T > &lhs, const [Vector](#)< T > &rhs)
- template<typename T>
bool [operator<=](#) (const [Vector](#)< T > &lhs, const [Vector](#)< T > &rhs)
- template<typename T>
bool [operator>](#) (const [Vector](#)< T > &lhs, const [Vector](#)< T > &rhs)
- template<typename T>
bool [operator>=](#) (const [Vector](#)< T > &lhs, const [Vector](#)< T > &rhs)

5.11.1 Function Documentation

5.11.1.1 [destroy_range\(\)](#)

```
template<typename T>
void destroy_range (
    T * first,
    T * last)
```

Definition at line 16 of file [Vector.h](#).

Here is the caller graph for this function:

5.11.1.2 [operator"!=\(\)](#)

```
template<typename T>
bool operator!= (
    const Vector< T > & lhs,
    const Vector< T > & rhs)
```

Definition at line 805 of file [Vector.h](#).

5.11.1.3 operator<()

```
template<typename T>
bool operator< (
    const Vector< T > & lhs,
    const Vector< T > & rhs)
```

Definition at line 810 of file [Vector.h](#).

Here is the call graph for this function:

5.11.1.4 operator<=()

```
template<typename T>
bool operator<= (
    const Vector< T > & lhs,
    const Vector< T > & rhs)
```

Definition at line 816 of file [Vector.h](#).

5.11.1.5 operator==(())

```
template<typename T>
bool operator==(
    const Vector< T > & lhs,
    const Vector< T > & rhs)
```

Definition at line 796 of file [Vector.h](#).

Here is the call graph for this function:

5.11.1.6 operator>()

```
template<typename T>
bool operator> (
    const Vector< T > & lhs,
    const Vector< T > & rhs)
```

Definition at line 821 of file [Vector.h](#).

5.11.1.7 operator>=()

```
template<typename T>
bool operator>= (
    const Vector< T > & lhs,
    const Vector< T > & rhs)
```

Definition at line 826 of file [Vector.h](#).

5.11.1.8 swap()

```
template<typename T>
void swap (
    Vector< T > & lhs,
    Vector< T > & rhs) [noexcept]
```

Definition at line 790 of file [Vector.h](#).

5.12 Vector.h

[Go to the documentation of this file.](#)

```
00001 #ifndef MY_VECTOR_H
00002 #define MY_VECTOR_H
00003
00004 #include <cstddef> // size_t, ptrdiff_t
00005 #include <stdexcept> // std::out_of_range, std::length_error
00006 #include <iterator> // std::reverse_iterator, iterator tags
00007 #include <memory> // std::uninitialized_copy, std::uninitialized_fill_n, std::destroy_n (C++17)
00008 // manual loop
00009 #include <utility> // std::move, std::swap, std::forward
00010 #include <initializer_list>
00011 #include <algorithm> // std::lexicographical_compare, std::equal, std::max
00012 #include <limits> // std::numeric_limits
00013 #include <type_traits> // std::is_nothrow_move_constructible, etc. (for potential optimization)
00014 // Helper function to destroy elements (needed before C++17's std::destroy_n)
00015 template<typename T>
00016 void destroy_range(T* first, T* last) {
00017     while (first != last) {
00018         first->~T();
00019         ++first;
00020     }
00021 }
00022
00023 template<typename T>
00024 class Vector {
00025 public:
00026     // --- Member Types ---
00027     using value_type = T;
00028     using pointer = T*;
00029     using const_pointer = const T*;
00030     using reference = T&
00031     using const_reference = const T&
00032     using size_type = std::size_t;
00033     using difference_type = std::ptrdiff_t;
00034
00035     // --- Iterators ---
00036     // Simple pointer-based iterators
00037     using iterator = T*;
00038     using const_iterator = const T*;
00039     using reverse_iterator = std::reverse_iterator<iterator>;
00040     using const_reverse_iterator = std::reverse_iterator<const_iterator>;
00041
00042     // --- Member Variables ---
00043 private:
00044     pointer data_ = nullptr; // Pointer to the beginning of the allocated buffer
00045     size_type size_ = 0; // Number of elements currently stored
00046     size_type capacity_ = 0; // Total capacity of the allocated buffer
00047     static inline int memory_reallocated = 0;
00048
00049     // --- Private Helper Functions ---
00050
00051     // Reallocates memory, moving existing elements. Strong exception guarantee desirable here.
00052     // For simplicity, this version offers basic guarantee (vector state might change on exception).
00053     void reallocate(size_type new_capacity) {
00054         if (new_capacity <= capacity_) return; // Should generally not happen if called correctly
00055
00056         // Use plain ::operator new to avoid potential T::operator new overloads interfering
00057         // Allocate raw memory first
00058         pointer new_data = static_cast<pointer> (::operator new(new_capacity * sizeof(value_type)));
00059
00060         // Move elements (or copy if move is not noexcept or not available)
00061         // Use uninitialized_move for potential performance gains and correctness
00062         // If move construction can throw, we might prefer copying or need more complex handling.
00063         // Let's simplify here and use move if possible, copy otherwise (basic guarantee).
```

```

00064         size_type elements_to_move = size_;
00065     try {
00066         if constexpr (std::is_nothrow_move_constructible_v<value_type>) {
00067             std::uninitialized_move(data_, data_ + size_, new_data);
00068         } else {
00069             std::uninitialized_copy(data_, data_ + size_, new_data); // Fallback to copy
00070         }
00071     } catch (...) {
00072         ::operator delete(new_data); // Clean up allocated memory on exception
00073         throw; // Re-throw the exception
00074     }
00075
00076     // Destroy old elements (if any)
00077     destroy_range(data_, data_ + size_);
00078
00079     // Deallocate old memory
00080     ::operator delete(data_);
00081
00082     // Update members
00083     data_ = new_data;
00084     capacity_ = new_capacity;
00085     // size_ remains the same
00086 }
00087
00088 // Calculates recommended new capacity (common strategy: double or 1.5x)
00089 size_type calculate_growth(size_type min_capacity) const {
00090     // Handle initial allocation and avoid overflow
00091     if (capacity_ > max_size() / 2) {
00092         return max_size();
00093     }
00094     return std::max(min_capacity, capacity_ + capacity_ / 2); // ~1.5x growth, minimum
00095 min_capacity
00096     // Or simpler doubling: return std::max(min_capacity, capacity_ == 0 ? 1 : capacity_ * 2);
00097 }
00098
00099 // Destroys elements and deallocates memory
00100 void clear_and_deallocate() {
00101     if (data_) {
00102         destroy_range(data_, data_ + size_); // Destroy elements
00103         ::operator delete(data_); // Deallocate buffer
00104         data_ = nullptr;
00105         size_ = 0;
00106         capacity_ = 0;
00107     }
00108 }
00109
00110 public:
00111     // --- Constructors ---
00112
00113     // Default constructor
00114     Vector() noexcept = default; // data_, size_, capacity_ already initialized to 0/nullptr
00115
00116     // Fill constructor (count default-inserted elements)
00117     explicit Vector(size_type count) : size_(count), capacity_(count) {
00118         if (count > max_size()) throw std::length_error("Vector(count): count exceeds max_size()");
00119         if (count == 0) return; // data_ remains nullptr
00120         data_ = static_cast<pointer> (::operator new(count * sizeof(value_type)));
00121         try {
00122             std::uninitialized_value_construct_n(data_, count); // C++17: Default constructs
00123             // Pre-C++17 manual loop:
00124             // for(size_type i = 0; i < count; ++i) { ::new (static_cast<void*>(data_ + i))
00125             value_type(); }
00126         } catch (...) {
00127             ::operator delete(data_);
00128             data_ = nullptr;
00129             capacity_ = 0;
00130             size_ = 0;
00131             throw;
00132         }
00133     }
00134
00135     // Fill constructor (count copies of value)
00136     Vector(size_type count, const_reference value) : size_(count), capacity_(count) {
00137         if (count > max_size()) throw std::length_error("Vector(count, value): count exceeds
00138         max_size()");
00139         if (count == 0) return;
00140         data_ = static_cast<pointer> (::operator new(count * sizeof(value_type)));
00141         try {
00142             std::uninitialized_fill_n(data_, count, value); // Copies value count times
00143         } catch (...) {
00144             ::operator delete(data_);
00145             data_ = nullptr;
00146             capacity_ = 0;
00147             size_ = 0;

```



```

00148         throw;
00149     }
00150 }
00151
00152 // Range constructor
00153 template <typename InputIt, typename = std::enable_if_t<
00154     std::is_base_of_v<std::input_iterator_tag, typename
00155     std::iterator_traits<InputIt>::iterator_category>
00156     >>
00157 Vector(InputIt first, InputIt last) {
00158     // Use reserve and push_back for input iterators
00159     // For random access iterators, could optimize by calculating distance first
00160     if constexpr (std::is_base_of_v<std::random_access_iterator_tag, typename
00161         std::iterator_traits<InputIt>::iterator_category>) {
00162         difference_type count = std::distance(first, last);
00163         if (count < 0) throw std::length_error("Vector range constructor: invalid range");
00164         size_type initial_size = static_cast<size_type>(count);
00165         if (initial_size > max_size()) throw std::length_error("Vector range constructor: range too
large");
00166         if (initial_size == 0) return;
00167         data_ = static_cast<pointer>(::operator new(initial_size * sizeof(value_type)));
00168         capacity_ = initial_size;
00169         try {
00170             std::uninitialized_copy(first, last, data_);
00171             size_ = initial_size;
00172         } catch (...) {
00173             ::operator delete(data_);
00174             data_ = nullptr;
00175             capacity_ = 0;
00176             size_ = 0;
00177             throw;
00178         }
00179     } else {
00180         // Input iterator path (less efficient, uses push_back)
00181         for (; first != last; ++first) {
00182             push_back(*first); // push_back handles allocation
00183         }
00184     }
00185 }
00186
00187 // Copy constructor
00188 Vector(const Vector& other) : size_(other.size_), capacity_(other.size_) { // Start with exact
capacity
00189     if (other.size_ == 0) return;
00190     if (other.size_ > max_size()) throw std::length_error("Vector copy constructor: source size
exceeds max_size()");
00191     data_ = static_cast<pointer>(::operator new(other.size_ * sizeof(value_type)));
00192     try {
00193         std::uninitialized_copy(other.data_, other.data_ + other.size_, data_);
00194     } catch (...) {
00195         ::operator delete(data_);
00196         data_ = nullptr;
00197         capacity_ = 0;
00198         size_ = 0;
00199         throw;
00200     }
00201 }
00202
00203 // Move constructor
00204 Vector(Vector&& other) noexcept
00205     : data_(other.data_), size_(other.size_), capacity_(other.capacity_)
00206 {
00207     // Leave the moved-from object in a valid, empty state
00208     other.data_ = nullptr;
00209     other.size_ = 0;
00210     other.capacity_ = 0;
00211 }
00212
00213 // Initializer list constructor
00214 Vector(std::initializer_list<T> init) : Vector(init.begin(), init.end()) {} // Delegate to range
constructor
00215
00216 // --- Destructor ---
00217 ~Vector() {
00218     clear_and_deallocate();
00219 }
00220
00221 // --- Assignment Operators ---
00222
00223 // Copy assignment operator (Copy-and-swap idiom for strong guarantee)
00224 Vector& operator=(const Vector& other) {
00225     if (this != &other) {
00226         Vector temp(other); // Use copy constructor (handles allocation and copy)
00227         swap(temp); // Swap internal state with the temporary
00228     }

```

```

00229         return *this;
00230     }
00231
00232     // Move assignment operator
00233     Vector& operator=(Vector&& other) noexcept {
00234         if (this != &other) {
00235             clear_and_deallocate(); // Destroy existing resources first
00236
00237             // Steal resources from other
00238             data_ = other.data_;
00239             size_ = other.size_;
00240             capacity_ = other.capacity_;
00241
00242             // Leave other in valid empty state
00243             other.data_ = nullptr;
00244             other.size_ = 0;
00245             other.capacity_ = 0;
00246         }
00247         return *this;
00248     }
00249
00250     // Initializer list assignment
00251     Vector& operator=(std::initializer_list<T> ilist) {
00252         assign(ilist.begin(), ilist.end()); // Use assign method
00253         return *this;
00254     }
00255
00256     // Assign (replace contents) - fill version
00257     void assign(size_type count, const T& value) {
00258         clear(); // Destroy existing elements but keep capacity if possible
00259         if (count > capacity_) {
00260             // Need to reallocate completely
00261             clear_and_deallocate(); // Get rid of old buffer
00262             if (count > max_size()) throw std::length_error("assign count exceeds max_size");
00263             data_ = static_cast<pointer>> (::operator new(count * sizeof(value_type)));
00264             capacity_ = count;
00265         }
00266         // Now fill the (potentially new) buffer
00267         try {
00268             std::uninitialized_fill_n(data_, count, value);
00269             size_ = count;
00270         } catch (...) {
00271             // If reallocation happened and fill fails, we need to clean up
00272             if (capacity_ == count) { // Indicates we just allocated
00273                 ::operator delete(data_);
00274                 data_ = nullptr;
00275                 capacity_ = 0;
00276             }
00277             size_ = 0; // Ensure size is 0 even if buffer wasn't deallocated
00278             throw;
00279         }
00280     }
00281
00282
00283     // Assign (replace contents) - range version
00284     template <typename InputIt, typename = std::enable_if_t<
00285         std::is_base_of_v<std::input_iterator_tag, typename
00286         std::iterator_traits<InputIt>::iterator_category>
00287     >>
00287     void assign(InputIt first, InputIt last) {
00288         clear(); // Destroy existing elements, keep buffer if possible
00289
00290         // Simple implementation: insert one by one
00291         // Could be optimized for random access iterators (calculate size, reserve, copy)
00292         for (; first != last; ++first) {
00293             push_back(*first); // push_back handles reallocation if needed
00294         }
00295     }
00296
00297     // Assign (replace contents) - initializer list version
00298     void assign(std::initializer_list<T> ilist) {
00299         assign(ilist.begin(), ilist.end());
00300     }
00301
00302     // --- Element Access ---
00303
00304     int getMemory_reallocated()
00305     {
00306         return memory_reallocated;
00307     }
00308
00309     // Access specified element with bounds checking
00310     reference at(size_type pos) {
00311         if (pos >= size_) {
00312             throw std::out_of_range("Vector::at: index out of range");
00313         }
00314         return data_[pos];

```

```

00315     }
00316     const_reference at(size_type pos) const {
00317         if (pos >= size_) {
00318             throw std::out_of_range("Vector::at: index out of range");
00319         }
00320         return data_[pos];
00321     }
00322
00323     // Access specified element (no bounds checking)
00324     reference operator[](size_type pos) {
00325         // Assert(pos < size_); // Optional assert for debug builds
00326         return data_[pos];
00327     }
00328     const_reference operator[](size_type pos) const {
00329         // Assert(pos < size_); // Optional assert for debug builds
00330         return data_[pos];
00331     }
00332
00333     // Access the first element
00334     reference front() {
00335         // Assert(!empty());
00336         return data_[0];
00337     }
00338     const_reference front() const {
00339         // Assert(!empty());
00340         return data_[0];
00341     }
00342
00343     // Access the last element
00344     reference back() {
00345         // Assert(!empty());
00346         return data_[size_ - 1];
00347     }
00348     const_reference back() const {
00349         // Assert(!empty());
00350         return data_[size_ - 1];
00351     }
00352
00353     // Direct access to the underlying array
00354     T* data() noexcept {
00355         return data_;
00356     }
00357     const T* data() const noexcept {
00358         return data_;
00359     }
00360
00361     // --- Iterators ---
00362
00363     iterator begin() noexcept {
00364         return data_;
00365     }
00366     const_iterator begin() const noexcept {
00367         return data_;
00368     }
00369     const_iterator cbegin() const noexcept {
00370         return data_;
00371     }
00372
00373     iterator end() noexcept {
00374         return data_ + size_;
00375     }
00376     const_iterator end() const noexcept {
00377         return data_ + size_;
00378     }
00379     const_iterator cend() const noexcept {
00380         return data_ + size_;
00381     }
00382
00383     reverse_iterator rbegin() noexcept {
00384         return reverse_iterator(end());
00385     }
00386     const_reverse_iterator rbegin() const noexcept {
00387         return const_reverse_iterator(end());
00388     }
00389     const_reverse_iterator crbegin() const noexcept {
00390         return const_reverse_iterator(cend());
00391     }
00392
00393     reverse_iterator rend() noexcept {
00394         return reverse_iterator(begin());
00395     }
00396     const_reverse_iterator rend() const noexcept {
00397         return const_reverse_iterator(begin());
00398     }
00399     const_reverse_iterator crend() const noexcept {
00400         return const_reverse_iterator(cbegin());
00401     }

```

```

00402
00403 // --- Capacity ---
00404
00405 // Checks whether the container is empty
00406 bool empty() const noexcept {
00407     return size_ == 0;
00408 }
00409
00410 // Returns the number of elements
00411 size_type size() const noexcept {
00412     return size_;
00413 }
00414
00415 // Returns the maximum possible number of elements
00416 size_type max_size() const noexcept {
00417     // Theoretical limit based on size_t and pointer difference type
00418     return std::numeric_limits<difference_type>::max();
00419     // Practical limit considering memory allocation overhead might be slightly less
00420     // return std::numeric_limits<size_type>::max() / sizeof(value_type); // Another common
approximation
00421 }
00422
00423
00424 // Increase the capacity of the vector (the total number of elements that the vector can hold
without requiring reallocation)
00425 void reserve(size_type new_cap) {
00426     if (new_cap > max_size()) {
00427         throw std::length_error("Vector::reserve capacity exceeds max_size()");
00428     }
00429     if (new_cap > capacity_) {
00430         reallocate(new_cap);
00431     }
00432     // Note: reserve never shrinks capacity in std::vector
00433 }
00434
00435
00436 // Returns the number of elements that the container has currently allocated space for
00437 size_type capacity() const noexcept {
00438     return capacity_;
00439 }
00440
00441 // Requests the removal of unused capacity
00442 void shrink_to_fit() {
00443     if (size_ < capacity_) {
00444         if (size_ == 0) {
00445             clear_and_deallocate(); // Special case: become empty
00446         } else {
00447             // Reallocate to exact size (or slightly more if needed)
00448             // Note: reallocate requires moving elements, so it's not noexcept
00449             Vector temp(std::move(*this)); // Move construct into temp (uses existing buffer)
00450             temp.reallocate(size_); // Reallocate temp to the target size
00451             swap(temp); // Swap back (this now has the shrunk buffer)
00452             // This approach avoids self-move assignment issues within reallocate
00453             // A direct reallocate(size_) call is simpler but needs care.
00454         }
00455     }
00456 }
00457
00458 // --- Modifiers ---
00459
00460 // Clears the contents (destroys elements, size becomes 0, capacity unchanged)
00461 void clear() noexcept {
00462     destroy_range(data_, data_ + size_);
00463     size_ = 0;
00464     // Capacity remains unchanged
00465 }
00466
00467 // Inserts elements
00468
00469 // Single element insert (copy)
00470 iterator insert(const_iterator pos, const T& value) {
00471     return emplace(pos, value); // Delegate to emplace
00472 }
00473
00474 // Single element insert (move)
00475 iterator insert(const_iterator pos, T&& value) {
00476     return emplace(pos, std::move(value)); // Delegate to emplace
00477 }
00478
00479 // Fill insert
00480 iterator insert(const_iterator pos, size_type count, const T& value) {
00481     if (count == 0) return iterator(const_cast<pointer>(pos)); // Non-const conversion needed
00482
00483     difference_type index = pos - cbegin();
00484     if (index < 0 || static_cast<size_type>(index) > size_) {
00485         throw std::out_of_range("Vector::insert iterator out of range");
00486     }

```

```

00487
00488     size_type current_size = size_; // Save old size
00489     size_type required_size = size_ + count;
00490     if (required_size > max_size()) throw std::length_error("Vector::insert count exceeds
max_size");
00491
00492     iterator insert_pos = begin() + index; // Modifiable iterator
00493
00494     if (required_size > capacity_) {
00495         // Reallocation needed
00496         size_type new_cap = calculate_growth(required_size);
00497         pointer new_data = static_cast<pointer> (::operator new(new_cap * sizeof(value_type)));
00498         iterator new_insert_pos = new_data + index;
00499         iterator new_end_pos = new_insert_pos + count;
00500
00501         // Try to construct the new elements first
00502         try {
00503             std::uninitialized_fill_n(new_insert_pos, count, value);
00504         } catch (...) {
00505             ::operator delete(new_data);
00506             throw;
00507         }
00508
00509         // Move elements before insertion point
00510         try {
00511             // Move construct elements before insertion point
00512             if constexpr (std::is_nothrow_move_constructible_v<value_type>) {
00513                 std::uninitialized_move(data_, insert_pos, new_data);
00514             } else {
00515                 std::uninitialized_copy(data_, insert_pos, new_data); // Fallback to copy
00516             }
00517         } catch (...) {
00518             destroy_range(new_insert_pos, new_end_pos); // Cleanup constructed new elements
00519             ::operator delete(new_data);
00520             throw;
00521         }
00522
00523         // Move elements after insertion point
00524         try {
00525             // Move construct elements after insertion point
00526             if constexpr (std::is_nothrow_move_constructible_v<value_type>) {
00527                 std::uninitialized_move(insert_pos, data_ + current_size, new_end_pos);
00528             } else {
00529                 std::uninitialized_copy(insert_pos, data_ + current_size, new_end_pos); //
Fallback to copy
00530             }
00531         } catch (...) {
00532             destroy_range(new_data, new_insert_pos); // Cleanup moved elements before insertion
point
00533             destroy_range(new_insert_pos, new_end_pos); // Cleanup constructed new elements
00534             ::operator delete(new_data);
00535             throw;
00536         }
00537
00538         // Deallocate old data
00539         destroy_range(data_, data_ + current_size);
00540         ::operator delete(data_);
00541
00542         // Update members
00543         data_ = new_data;
00544         size_ = required_size;
00545         capacity_ = new_cap;
00546         return new_insert_pos;
00547     } else {
00548         // No reallocation needed, shift elements within the existing buffer
00549         iterator old_end = end();
00550         size_type elements_to_move = old_end - insert_pos; // number of elements from pos to end
00551         iterator new_end_target = old_end + count;
00552
00553         if (elements_to_move > 0) {
00554             // Move construct elements into the newly available space at the end
00555             std::uninitialized_move(old_end - std::min(count, elements_to_move), old_end,
new_end_target - std::min(count, elements_to_move));
00556
00557             // Move assign the remaining overlapping elements (or copy if move assign not safe)
00558             std::move_backward(insert_pos, old_end - std::min(count, elements_to_move), old_end);
00559
00560             // Destroy the elements that were moved from but not overwritten by insert
00561             // This happens if count < elements_to_move
00562             // Note: This destruction part is tricky and often overlooked. A simpler approach
might destroy all moved-from elements
00563             // and then fill, but that's less efficient. Let's assume move ops handle this or
accept potential issues in this simplified impl.
00564         }
00565     }
00566
00567

```

```

00568
00569         // Insert the new elements by copy constructing/assigning
00570         std::fill_n(insert_pos, count, value);
00571
00572
00573         size_ = required_size;
00574         return insert_pos;
00575     }
00576 }
00577
00578 // Range insert - Omitted for brevity, complex implementation
00579
00580
00581 // Emplace element (construct in-place)
00582 template <typename... Args>
00583 iterator.emplace(const_iterator pos, Args&&... args) {
00584     difference_type index = pos - cbegin();
00585     if (index < 0 || static_cast<size_type>(index) > size_) {
00586         throw std::out_of_range("Vector::emplace iterator out of range");
00587     }
00588
00589     if (size_ == capacity_) {
00590         // Reallocation required
00591         size_type new_cap = calculate_growth(size_ + 1);
00592         pointer new_data = static_cast<pointer> (::operator new(new_cap * sizeof(value_type)));
00593         iterator insert_pos = new_data + index;
00594
00595         // Construct the new element first in the new buffer
00596         try {
00597             ::new (static_cast<void*>(insert_pos)) value_type(std::forward<Args>(args)...);
00598         } catch (...) {
00599             ::operator delete(new_data); // Clean up allocated memory
00600             throw;
00601         }
00602
00603         // Move elements before insertion point
00604         try {
00605             if constexpr (std::is_nothrow_move_constructible_v<value_type>) {
00606                 std::uninitialized_move(data_, data_ + index, new_data);
00607             } else {
00608                 std::uninitialized_copy(data_, data_ + index, new_data);
00609             }
00610         } catch (...) {
00611             insert_pos->~T(); // Destroy the newly constructed element
00612             ::operator delete(new_data);
00613             throw;
00614         }
00615
00616         // Move elements after insertion point
00617         try {
00618             if constexpr (std::is_nothrow_move_constructible_v<value_type>) {
00619                 std::uninitialized_move(data_ + index, data_ + size_, insert_pos + 1);
00620             } else {
00621                 std::uninitialized_copy(data_ + index, data_ + size_, insert_pos + 1);
00622             }
00623         } catch (...) {
00624             destroy_range(new_data, insert_pos); // Destroy elements before insertion point
00625             insert_pos->~T(); // Destroy the newly constructed element
00626             ::operator delete(new_data);
00627             throw;
00628         }
00629
00630         // Clean up old data
00631         destroy_range(data_, data_ + size_);
00632         ::operator delete(data_);
00633
00634         // Update members
00635         data_ = new_data;
00636         capacity_ = new_cap;
00637         size_++;
00638         return insert_pos; // Return iterator to the newly emplaced element
00639     } else {
00640         // No reallocation, shift elements within the buffer
00641         iterator insert_pos = begin() + index; // Modifiable iterator
00642         if (insert_pos == end()) {
00643             // Emplacing at the end is simpler
00644             ::new (static_cast<void*>(data_ + size_)) value_type(std::forward<Args>(args)...);
00645         } else {
00646             // Need to shift elements to make space
00647             // Move construct the last element into the uninitialized space
00648             ::new (static_cast<void*>(data_ + size_)) value_type(std::move(back()));
00649             // Move assign elements backwards
00650             std::move_backward(insert_pos, end() - 1, end());
00651             // Overwrite/assign at the insertion point (or destroy and construct)
00652             *insert_pos = value_type(std::forward<Args>(args)...); // Simpler: Assign (requires
copy/move assignable)

```

```

00654         // Alternative: insert_pos->~T(); ::new (static_cast<void*>(insert_pos))
value_type(std::forward<Args>(args)...);
00655     }
00656     size_++;
00657     return insert_pos;
00658 }
00659 }
00660
00661 // Erases elements
00662 iterator erase(const_iterator pos) {
00663     difference_type index = pos - cbegin();
00664     if (index < 0 || static_cast<size_type>(index) >= size_) {
00665         throw std::out_of_range("Vector::erase iterator out of range");
00666     }
00667
00668     iterator erase_pos = begin() + index;
00669
00670     // Move elements after the erased position one step back
00671     std::move(erase_pos + 1, end(), erase_pos);
00672
00673     // Destroy the now-duplicate element at the end
00674     destroy_range(data_ + size_ - 1, data_ + size_);
00675
00676     size_--;
00677     return erase_pos; // Iterator following the last removed element
00678 }
00679
00680
00681 // Range erase - Omitted for brevity
00682 iterator erase(const_iterator first, const_iterator last) {
00683     difference_type first_idx = first - cbegin();
00684     difference_type last_idx = last - cbegin();
00685
00686     if (first_idx < 0 || static_cast<size_type>(first_idx) > size_ ||
00687         last_idx < first_idx || static_cast<size_type>(last_idx) > size_) {
00688         throw std::out_of_range("Vector::erase range iterators out of range");
00689     }
00690
00691     if (first == last) return iterator(const_cast<pointer>(first)); // Nothing to erase
00692
00693     iterator erase_first = begin() + first_idx;
00694     iterator erase_last = begin() + last_idx;
00695
00696     // Move elements from 'last' to the end, overwriting the erased range
00697     iterator new_end = std::move(erase_last, end(), erase_first);
00698
00699     // Destroy the elements that were moved from (now at the end of the used part)
00700     destroy_range(new_end, end());
00701
00702     size_ -= (last_idx - first_idx);
00703
00704     return erase_first; // Iterator to the first element after the erased range
00705 }
00706
00707 // Appends the given element value to the end of the container
00708 void push_back(const T& value) {
00709     emplace_back(value); // Delegate to emplace_back
00710 }
00711 void push_back(T&& value) {
00712     emplace_back(std::move(value)); // Delegate to emplace_back (move version)
00713 }
00714
00715 // Appends a new element to the end of the container (constructed in-place)
00716 template <typename... Args>
00717 reference emplace_back(Args&&... args) {
00718     if (size_ == capacity_) {
00719         memory_reallocated++;
00720         reserve(calculate_growth(size_ + 1)); // Reserve handles reallocation
00721     }
00722     // Construct in place at the end
00723     ::new (static_cast<void*>(data_ + size_)) value_type(std::forward<Args>(args)...);
00724     size_++;
00725     return back(); // Return reference to the newly added element
00726 }
00727
00728 // Removes the last element of the container
00729 void pop_back() {
00730     if (!empty()) {
00731         size_--;
00732         data_[size_].~T(); // Destroy the last element
00733     }
00734     // else: Undefined behavior like std::vector? Or just do nothing? Doing nothing is safer.
00735 }
00736
00737 // Resizes the container to contain count elements
00738 void resize(size_type count) {
00739     if (count < size_) {

```

```

00740         // Shrink: destroy elements from count to size_
00741         destroy_range(data_ + count, data_ + size_);
00742         size_ = count;
00743     } else if (count > size_) {
00744         // Grow: reserve if needed, then default-construct new elements
00745         if (count > capacity_) {
00746             reserve(calculate_growth(count)); // Use growth strategy
00747         }
00748         // Default construct elements from size_ to count
00749         std::uninitialized_value_construct_n(data_ + size_, count - size_);
00750         // Pre-C++17 loop:
00751         // for(size_t i = size_; i < count; ++i) { ::new (static_cast<void*>(data_ + i))
value_type(); }
00752         size_ = count;
00753     }
00754     // if count == size_, do nothing
00755 }
00756
00757 // Resizes the container to contain count elements (copying value if growing)
00758 void resize(size_type count, const value_type& value) {
00759     if (count < size_) {
00760         // Shrink: destroy elements from count to size_
00761         destroy_range(data_ + count, data_ + size_);
00762         size_ = count;
00763     } else if (count > size_) {
00764         // Grow: reserve if needed, then copy-construct new elements
00765         if (count > capacity_) {
00766             reserve(calculate_growth(count)); // Use growth strategy
00767         }
00768         // Copy construct elements from size_ to count using 'value'
00769         std::uninitialized_fill_n(data_ + size_, count - size_, value);
00770         size_ = count;
00771     }
00772     // if count == size_, do nothing
00773 }
00774
00775 // Exchanges the contents of the container with those of other
00776 void swap(Vector& other) noexcept {
00777     using std::swap; // Enable ADL
00778     swap(data_, other.data_);
00779     swap(size_, other.size_);
00780     swap(capacity_, other.capacity_);
00781 }
00782
00783 }; // End class Vector
00784
00785 // --- Non-member Function Overloads ---
00786
00787 // Non-member swap
00788 template <typename T>
00789 void swap(Vector<T>& lhs, Vector<T>& rhs) noexcept {
00790     lhs.swap(rhs);
00791 }
00792
00793 // Comparison operators
00794 template <typename T>
00795 bool operator==(const Vector<T>& lhs, const Vector<T>& rhs) {
00796     if (lhs.size() != rhs.size()) {
00797         return false;
00798     }
00799     // Use std::equal with the iterators
00800     return std::equal(lhs.begin(), lhs.end(), rhs.begin());
00801 }
00802
00803 template <typename T>
00804 bool operator!=(const Vector<T>& lhs, const Vector<T>& rhs) {
00805     return !(lhs == rhs);
00806 }
00807
00808 template <typename T>
00809 bool operator<(const Vector<T>& lhs, const Vector<T>& rhs) {
00810     // Use std::lexicographical_compare with the iterators
00811     return std::lexicographical_compare(lhs.begin(), lhs.end(), rhs.begin(), rhs.end());
00812 }
00813
00814 template <typename T>
00815 bool operator>(const Vector<T>& lhs, const Vector<T>& rhs) {
00816     return !(rhs < lhs);
00817 }
00818
00819 template <typename T>
00820 bool operator>=(const Vector<T>& lhs, const Vector<T>& rhs) {
00821     return rhs < lhs;
00822 }
00823
00824 template <typename T>

```



```
00826 bool operator>=(const Vector<T>& lhs, const Vector<T>& rhs) {  
00827     return !(lhs < rhs);  
00828 }  
00829  
00830  
00831 #endif // MY_VECTOR_H
```


Index

- ~Vector
 - Vector< T >, [20](#)
- ~duom
 - duom, [8](#)
- abstractClassFunction
 - duom, [9](#)
 - human, [13](#)
- addPazymiai
 - duom, [9](#)
- assign
 - Vector< T >, [21](#)
- at
 - Vector< T >, [21](#)
- average
 - program.cpp, [32](#)
 - template.h, [46](#)
- back
 - Vector< T >, [21](#), [22](#)
- begin
 - Vector< T >, [22](#)
- capacity
 - Vector< T >, [22](#)
- cbegin
 - Vector< T >, [22](#)
- cend
 - Vector< T >, [22](#)
- check_menu
 - program.cpp, [32](#)
 - template.h, [46](#)
- class_write_test
 - program.cpp, [32](#)
 - template.h, [46](#)
- clear
 - Vector< T >, [23](#)
- compare
 - program.cpp, [33](#)
 - template.h, [46](#)
- const_iterator
 - Vector< T >, [17](#)
- const_pointer
 - Vector< T >, [17](#)
- const_reference
 - Vector< T >, [18](#)
- const_reverse_iterator
 - Vector< T >, [18](#)
- crbegin
 - Vector< T >, [23](#)
- createSampleStudent
 - StudentaiTest, [14](#)
- crend
 - Vector< T >, [23](#)
- data
 - Vector< T >, [23](#)
- destroy_range
 - Vector.h, [53](#)
- difference_type
 - Vector< T >, [18](#)
- duom, [7](#)
 - ~duom, [8](#)
 - abstractClassFunction, [9](#)
 - addPazymiai, [9](#)
 - duom, [8](#), [9](#)
 - getExam, [9](#)
 - getMark, [9](#)
 - getPav, [10](#)
 - getPazymiai, [10](#)
 - getPazymiai_at, [10](#)
 - getVar, [10](#)
 - getVid_med, [10](#)
 - isClear, [10](#)
 - operator<<, [12](#)
 - operator>>, [12](#)
 - operator=, [10](#), [11](#)
 - operator==, [11](#), [12](#)
 - setExam, [11](#)
 - setMark, [11](#)
 - setPazymiai, [11](#)
 - setVid_med, [11](#)
- emplace
 - Vector< T >, [23](#)
- emplace_back
 - Vector< T >, [24](#)
- empty
 - Vector< T >, [24](#)
- end
 - Vector< T >, [24](#)
- erase
 - Vector< T >, [24](#), [25](#)
- exam
 - temp, [15](#)
- front
 - Vector< T >, [25](#)
- getExam

- duom, 9
- getMark
 - duom, 9
- getMemory_relocated
 - Vector< T >, 25
- getPav
 - duom, 10
- getPazymiai
 - duom, 10
- getPazymiai_at
 - duom, 10
- getVar
 - duom, 10
- getVid_med
 - duom, 10
- human, 12
 - abstractClassFunction, 13
 - pav, 13
 - var, 13
- insert
 - Vector< T >, 25, 26
- isClear
 - duom, 10
- iterator
 - Vector< T >, 18
- main
 - main.cpp, 31
- main.cpp, 31
 - main, 31
- mark
 - temp, 15
- max_size
 - Vector< T >, 26
- median
 - program.cpp, 33
 - template.h, 46
- menu
 - program.cpp, 33
 - template.h, 46
- method_test
 - program.cpp, 33
 - template.h, 47
- operator!=
 - Vector.h, 53
- operator<
 - Vector.h, 53
- operator<<
 - duom, 12
- operator<=
 - Vector.h, 54
- operator>
 - Vector.h, 54
- operator>>
 - duom, 12
- operator>=
 - Vector.h, 54
- operator=
 - duom, 10, 11
 - Vector< T >, 26
- operator==
 - duom, 11, 12
 - Vector.h, 54
- operator[]
 - Vector< T >, 27
- pav
 - human, 13
 - temp, 15
- pazymiai
 - temp, 15
- pointer
 - Vector< T >, 18
- pop_back
 - Vector< T >, 27
- print_answers_console
 - program.cpp, 33
 - template.h, 47
- print_answers_to_file
 - program.cpp, 33
 - template.h, 47
- print_data_to_file
 - program.cpp, 34
 - template.h, 47
- program.cpp, 32
 - average, 32
 - check_menu, 32
 - class_write_test, 32
 - compare, 33
 - median, 33
 - menu, 33
 - method_test, 33
 - print_answers_console, 33
 - print_answers_to_file, 33
 - print_data_to_file, 34
 - random_grades, 34
 - random_names_grades, 34
 - read_file, 34
 - read_from_console, 34
 - read_names_from_console, 35
 - sorting, 35
 - vardai, 35
 - vid_med_calc, 35
- push_back
 - Vector< T >, 27
- random_grades
 - program.cpp, 34
 - template.h, 47
- random_names_grades
 - program.cpp, 34
 - template.h, 48
- rbegin
 - Vector< T >, 28
- read_file

- program.cpp, 34
 - template.h, 48
- read_from_console
 - program.cpp, 34
 - template.h, 48
- read_names_from_console
 - program.cpp, 35
 - template.h, 48
- reference
 - Vector< T >, 18
- rend
 - Vector< T >, 28
- reserve
 - Vector< T >, 28
- resize
 - Vector< T >, 29
- reverse_iterator
 - Vector< T >, 18
- setExam
 - duom, 11
- setMark
 - duom, 11
- setPazymiai
 - duom, 11
- SetUp
 - StudentaiTest, 14
- setVid_med
 - duom, 11
- shrink_to_fit
 - Vector< T >, 29
- size
 - Vector< T >, 29
- size_type
 - Vector< T >, 19
- sorting
 - program.cpp, 35
 - template.h, 48
- std.h, 43
- Studentai_tests.cpp, 43
- StudentaiTest, 13
 - createSampleStudent, 14
 - SetUp, 14
 - testNd, 14
- swap
 - Vector< T >, 29
 - Vector.h, 54
- temp, 14
 - exam, 15
 - mark, 15
 - pav, 15
 - pazymiai, 15
 - var, 15
 - vid_med, 15
- template.h, 45
 - average, 46
 - check_menu, 46
 - class_write_test, 46
 - compare, 46
 - median, 46
 - menu, 46
 - method_test, 47
 - print_answers_console, 47
 - print_answers_to_file, 47
 - print_data_to_file, 47
 - random_grades, 47
 - random_names_grades, 48
 - read_file, 48
 - read_from_console, 48
 - read_names_from_console, 48
 - sorting, 48
 - test_file_location, 49
 - vid_med_calc, 49
- test_file_location
 - template.h, 49
- testNd
 - StudentaiTest, 14
- value_type
 - Vector< T >, 19
- var
 - human, 13
 - temp, 15
- vardai
 - program.cpp, 35
- Vector
 - Vector< T >, 19, 20
- Vector< T >, 16
 - ~Vector, 20
 - assign, 21
 - at, 21
 - back, 21, 22
 - begin, 22
 - capacity, 22
 - cbegin, 22
 - cend, 22
 - clear, 23
 - const_iterator, 17
 - const_pointer, 17
 - const_reference, 18
 - const_reverse_iterator, 18
 - crbegin, 23
 - crend, 23
 - data, 23
 - difference_type, 18
 - emplace, 23
 - emplace_back, 24
 - empty, 24
 - end, 24
 - erase, 24, 25
 - front, 25
 - getMemory_reallocated, 25
 - insert, 25, 26
 - iterator, 18
 - max_size, 26
 - operator=, 26
 - operator[], 27

- pointer, [18](#)
- pop_back, [27](#)
- push_back, [27](#)
- rbegin, [28](#)
- reference, [18](#)
- rend, [28](#)
- reserve, [28](#)
- resize, [29](#)
- reverse_iterator, [18](#)
- shrink_to_fit, [29](#)
- size, [29](#)
- size_type, [19](#)
- swap, [29](#)
- value_type, [19](#)
- Vector, [19](#), [20](#)
- Vector.h, [52](#)
 - destroy_range, [53](#)
 - operator!=, [53](#)
 - operator<, [53](#)
 - operator<=, [54](#)
 - operator>, [54](#)
 - operator>=, [54](#)
 - operator==, [54](#)
 - swap, [54](#)
- vid_med
 - temp, [15](#)
- vid_med_calc
 - program.cpp, [35](#)
 - template.h, [49](#)